PROJECT KNOWLEDGE & INTERVIEW GUIDE

FoodLink / FoodBridge-AI — UCS503P (2026-27 ODD)

> **How to read this document.** Every claim below was checked against the code in this repository on 2026-09-01. Where something does **not** exist, it says so explicitly rather than describing what a project like this usually has. Weaknesses are named, not hidden — being able to name the weak part of your own system is the single strongest signal in an interview.

> Verification performed: all 37 backend tests were executed and pass (`pytest code/tests` → `37 passed`). Security claims were checked by grep, not assumed.

---

1. Executive Overview

1.1 In plain language

Restaurants, canteens, and event caterers routinely end the day with edible food they cannot sell. A few kilometres away, community kitchens and shelters are feeding people who need exactly that food. The two sides rarely connect in time, because surplus food has a deadline measured in hours, and phoning around for a taker is slow.

FoodLink is a coordination platform that closes that gap. A donor posts surplus food with a location and a pickup deadline. The server immediately ranks nearby verified recipient organisations and attaches a match score. A recipient accepts, a volunteer courier claims the pickup, collects, and delivers. Every one of those steps is timestamped by the server, so the platform can prove how fast it actually moved food — not just claim it did.

The word "AI" in the repository name is worth being precise about in an interview: **the matching engine is an explainable weighted-sum heuristic, not a machine-learning model.** That was a deliberate choice (see §17), and the code says so in its own comments. Claiming a trained model exists would be the fastest way to fail a technical interview on this project.

1.2 The problem it solves

Three specific problems, in order of how much the code cares about them:

1. **Time-to-claim.** Surplus food expires. The delay between "I have food" and "someone has agreed to take it" is the delay that decides whether the rest of the chain can finish at all. This is the platform's primary reported metric.
2. **Discovery.** A donor does not know which of forty kitchens can use 200 meals of refrigerated biryani within the next three hours. The ranking engine answers that automatically.
3. **Accountability.** Self-reported impact numbers are worthless. Because every status transition is stamped server-side into an append-only table, the rescue rate and time-to-claim figures are derived from evidence.

1.3 Target users — the four roles

The system has exactly four roles, defined in `UserRole` (models.py:63):

Role	Who they are	What they do
`donor`	Restaurant, canteen, caterer, individual	Posts surplus food, can cancel their own donation
`ngo`	Community kitchen, shelter, NGO	Accepts donations, posts standing requirements, marks completion
`volunteer`	Courier	Claims a pickup, marks picked-up and delivered
`admin`	Platform operator	Verifies organisations, suspends accounts, creates any account, runs expiry sweep

A critical detail worth memorising: only three of these four roles can be self-registered. `SELF_SIGNUP_ROLES` (models.py:76) deliberately excludes `admin`, and `RegisterRequest._reject_self_made_admins` (schemas.py:45) enforces it at the schema layer.

The first administrator can only be created from the command line (`python -m foodlink.cli create-admin`). This is one of the best design decisions in the project and a near-certain interview question.

1.4 Main features (what actually exists)

- Email/password registration and login with JWT bearer tokens
- Four role-specific web portals with route guards
- A **separate mobile UI** (26 files under `frontend/src/mobile/`) served at its own URL space `/m/*` — the same portals in a phone-shaped layout
- Donation posting with geographic coordinates and an absolute pickup deadline
- **Automatic recipient ranking** on post, with a per-criterion explanation
- A **nine-state donation lifecycle** with a server-enforced transition table
- Append-only status history driving all metrics
- Standing requirements posted by recipients (demand visible before supply)
- Organisation verification workflow (admins vouch; unverified cannot accept)
- Platform metrics: time-to-claim, handover time, rescue rate, expiry loss rate
- Admin console: user list, suspend/restore/re-role, verify/revoke, expiry sweep
- An administrative CLI for bootstrapping and account recovery

1.5 What does NOT exist (state this honestly)

Verified absent by inspection and grep:

- **No Docker / docker-compose** anywhere in the repository
- **No CI for tests.** The only workflow is `.github/workflows/mkdocs.yml`, which builds and deploys documentation. Tests never run automatically.
- **No database migrations.** No Alembic. Schema is created by `Base.metadata.create_all` at startup (main.py:25).
- **No frontend tests.** Zero. No Jest, Vitest, Playwright, or Cypress.
- **No rate limiting** (grep for `slowapi`/`limiter` → nothing)
- **No refresh tokens** (grep → nothing). One long-lived access token only.
- **No file upload endpoint.** `imageUrl` is a text column; there is no `UploadFile` anywhere. Images arrive as base64/URL strings.
- **No WebSockets, no background jobs, no queues, no cron.** The expiry sweep is an HTTP endpoint someone must call.
- **No email sending, no SMS, no push notifications, no payment provider, no third-party API of any kind.** The system has zero external service dependencies.
- **No deployment configuration.** No Procfile, no nginx config, no Kubernetes manifests, no cloud config.
- **No `.env` file committed** (correctly — `.gitignore` covers `.env`).

Stale artefact: `code/Makefile` is leftover C++ scaffolding from the university template (it compiles `libbvr_math.so` from `src/lib/Bvr/Math/`). It has nothing to do with FoodLink. `code/src/` contains no Python. If an interviewer opens that file, say so plainly — it is template residue.

1.6 Technology stack, and why each piece

Backend

Technology	Version	Why it is here
Python	≥3.8 declared, 3.10+ syntax used	`X | None` union syntax throughout means it actually needs 3.10+ — the `requires-python = ">=3.8"` in `pyproject.toml` is inherited from the template and is **wrong**.

FastAPI	≥0.115	Gives dependency injection (`Depends`), automatic request validation from type hints, and a free OpenAPI/Swagger UI at `/docs`. The DI system is what makes `require_roles(...)` composable.
SQLAlchemy	≥2.0	Modern typed ORM (`Mapped[...]`, `mapped_column`). Parameterises every query, which is why SQL injection is structurally absent. Also lets the same code target SQLite and Postgres.
Pydantic	≥2.9	Validates and serialises every request/response. `alias_generator=to_camel` bridges Python `snake_case` and TypeScript `camelCase` in one place.
PyJWT	≥2.9	Signs and verifies HS256 access tokens.
bcrypt	≥4.2	Password hashing with a per-password salt and deliberate slowness.
uvicorn	≥0.30	ASGI server that actually runs the app.
python-multipart	≥0.0.9	Required because the login endpoint accepts OAuth2 form-encoded fields, not JSON.
pytest + httpx	≥8.3 / ≥0.27	Test runner and the transport behind FastAPI's `TestClient`.

Frontend

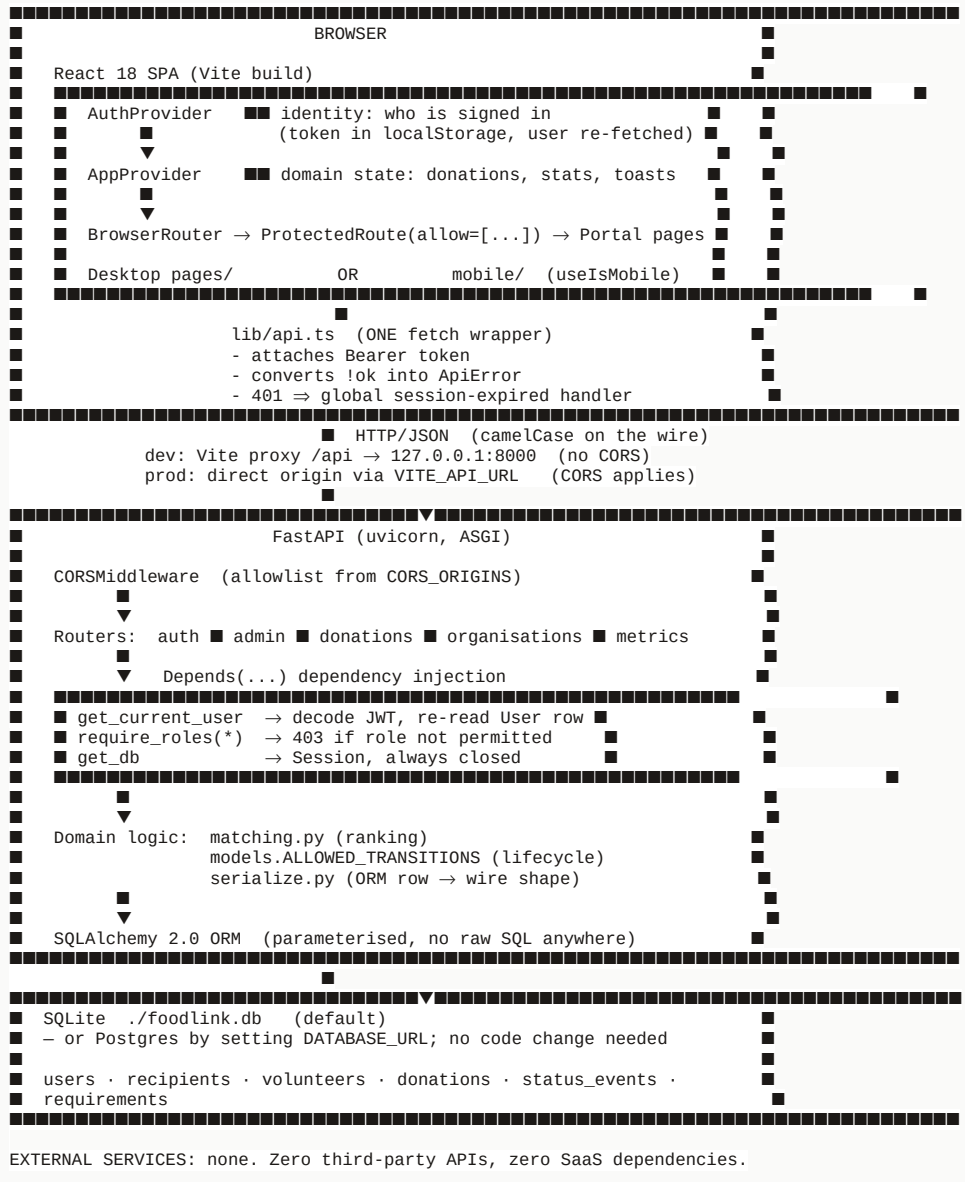
Technology	Version	Why it is here
React	18.3	Component model; the whole UI is function components with hooks.
TypeScript	5.5	The wire types in `lib/api.ts` mirror the Pydantic schemas, so a backend field rename becomes a compile error rather than a runtime `undefined`.
Vite	5.4	Dev server with HMR, and — importantly — a **proxy** that forwards `/api` to `:8000` so development has no CORS negotiation at all.
React Router	6.26	Nested routes; `<Outlet/>` is what makes `ProtectedRoute` a wrapper rather than a per-page check.
Tailwind CSS	3.4	Utility classes. The palette is **retuned** in `tailwind.config.js` — warm stone/moss/clay instead of default cool grays and neon emerald — so the app does not look like generic template SaaS.
lucide-react	0.441	Icon set.

Notice what is absent from the frontend: no Redux, no Zustand, no React Query, no Axios, no form library, no component library. State is React Context; HTTP is `fetch` wrapped in one module. For an application of this size that is a defensible choice, and §17 explains how to defend it.

---

2. Complete Architecture

2.1 The system at a glance




```
EXTERNAL SERVICES: none. Zero third-party APIs, zero SaaS dependencies.
```

2.2 Frontend architecture

Two providers wrap everything, and the split is deliberate (AuthContext.tsx:1):

```
<AuthProvider>          identity — must settle BEFORE data is fetched
 <AppProvider>         domain data — only meaningful once identity exists
   <BrowserRouter>
     <Routes>
       public:     /  and  /login
       guarded:    <ProtectedRoute allow={['donor','admin']}>
                     <DonorLayout>          ← chrome: sidebar/navbar
                       <Outlet/>            ← the actual page
```

Why the split matters: identity decides what the app is *allowed* to fetch, so it has to resolve first. And signing out clears identity without having to reason about half-loaded donation state.

Routing map (verified from `App.tsx`):

Path prefix	Roles allowed	Layout
`/` , `/login`	public	none
`/donor/*`	`donor`, `admin`	`DonorLayout`
`/ngo/*`	`ngo`, `admin`	`NGOLayout`

`/volunteer/*`	`volunteer`, `admin`	`VolunteerLayout`
`/admin/*`	`admin`	`AdminLayout`
`/m/*`	any signed-in (inner `MobileRole` guard per portal)	`MobileShell`
`*`	—	redirects to `/`

Note `admin` appears in every `allow` list. That is intentional and documented in the file: an admin can act on any donation through the API anyway, and seeing what a donor sees is most of what support work is.

2.3 Backend architecture

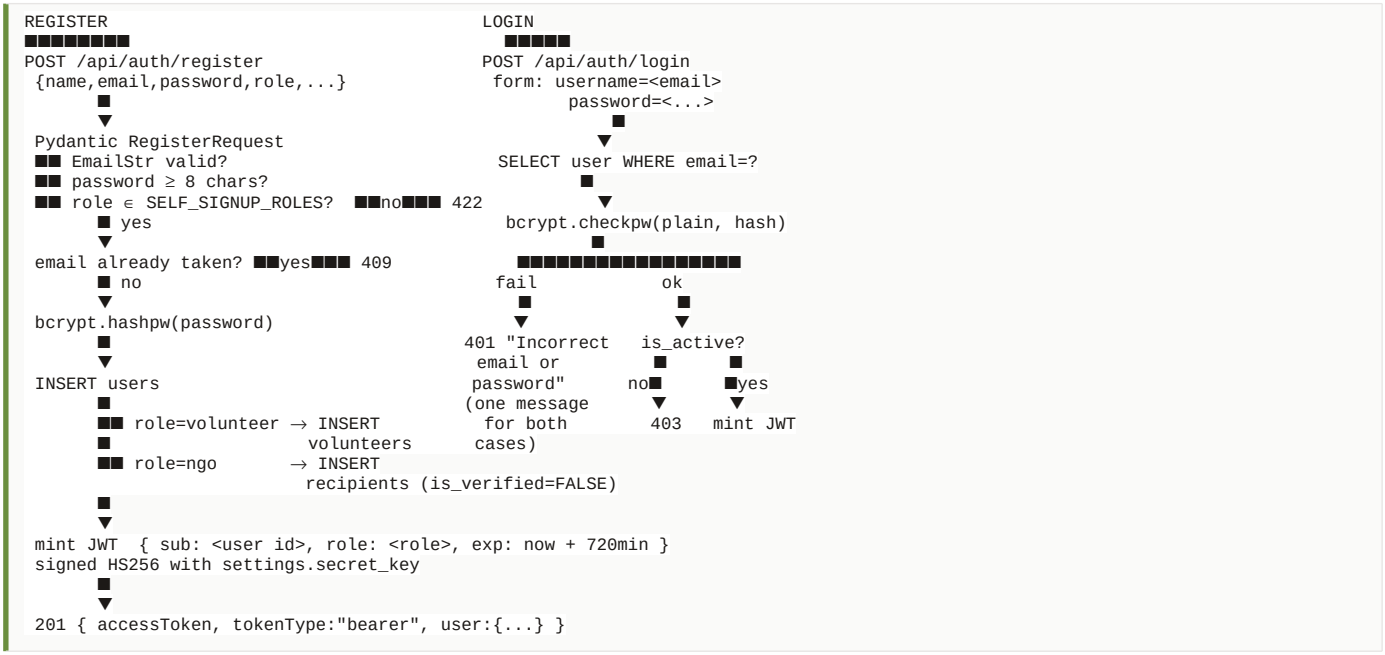
The backend is a **three-layer application, not a strict Controller→Service→ Repository stack.** Be precise about this in an interview:

```
Router (FastAPI path function)
 ■■ receives a Pydantic-validated body      ← validation layer
 ■■ receives User via Depends(require_roles) ← auth/authz layer
 ■■ contains the business rules inline       ←  no separate service layer
 ■■ talks to SQLAlchemy Session directly     ←  no repository layer

Shared domain modules pulled out where reuse demanded it:
 matching.py    — pure functions, no DB, fully unit-testable
 serialize.py   — ORM → wire shape
 security.py    — hashing, token mint/verify, role dependency
 models.py      — tables AND the lifecycle rules (ALLOWED_TRANSITIONS)
```

This is a real architectural observation, and a good one to volunteer yourself: there is no service layer. `update_status` in donations.py:165 is ~80 lines that validate the transition, check the role, resolve the recipient, guard the courier race, increment counters, append the event, and commit. In a larger system that belongs in `DonationService.transition()`. At this size, the indirection would cost more than it buys — but you should be able to say *where* you would cut it if the project grew (§17).

2.4 Authentication flow




```
EVERY SUBSEQUENT AUTHENTICATED REQUEST
Authorization: Bearer <jwt>
OAuth2PasswordBearer extracts the token
get_current_user()                    [security.py:41]
 jwt.decode(token, secret, HS256)
   bad signature / expired / malformed  401
   ok
 user = db.get(User, int(payload["sub"]))   DB READ EVERY REQUEST
 user is None  OR  not user.is_active       401
   (endpoints that need a specific role)
require_roles(UserRole.ngo, UserRole.admin)    [security.py:62]
 user.role not in roles  403
path function runs
```

The single most important subtlety here: the token carries `role`, but `get_current_user` **ignores it** and re-reads the `User` row from the database on every request. That costs one indexed primary-key lookup per request and buys something valuable — an administrator suspending an account takes effect *immediately*, mid-session, rather than whenever the token happens to expire. The frontend is built around this (`setUnauthorizedHandler`). This is a guaranteed interview question; §7 and §21 both drill it.

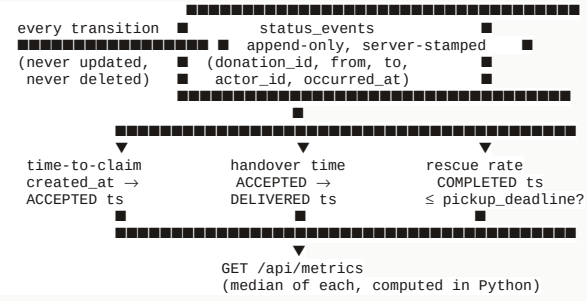
2.5 Request / response flow — a concrete trace

Taking "an NGO accepts a donation", the highest-value path in the system:

```
NGO clicks "Accept" on a donation card
  ■
  ▼
useAction().run('accept-42', () => updateDonationStatus('42','ACCEPTED'))
  ■   sets pendingKey='accept-42' → only THAT row shows a spinner
  ▼
AppContext.updateDonationStatus                [AppContext.tsx]
  ▼
api.updateStatus(42, 'ACCEPTED', {})           [lib/api.ts:398]
  ▼
POST /api/donations/42/status
Authorization: Bearer <jwt>
{"status":"ACCEPTED"}
  ■
  ▼■■■■■■■■■■■■ network / Vite proxy ■■■■■■■■■■■■■
  ■
  ▼
FastAPI routes to update_status()              [donations.py:165]
  ■
  ■■ Depends(get_db)            → Session
  ■■ Depends(get_current_user)  → User (JWT decoded, row re-read, active?)
  ■
  ■■ _get_or_404(db, 42)
  ■     SELECT donation + selectinload(donor, recipient, volunteer→user, events)
  ■     ■■ not found ■■■ 404
  ■
  ■■ TRANSITION LEGAL?
  ■     ACCEPTED ∈ ALLOWED_TRANSITIONS[donation.status]?
  ■     ■■ no ■■■ 409 "Cannot move a donation from X to ACCEPTED"
  ■
  ■■ ROLE PERMITTED?
  ■     user.role ∈ TRANSITION_ROLES[ACCEPTED] = {ngo, admin}?
  ■     ■■ no ■■■ 403 "Your role cannot set a donation to ACCEPTED"
  ■
  ■■ SIDE EFFECTS for ACCEPTED:
  ■     ■■ resolve recipient:
  ■     ■    admin → db.get(Recipient, body.recipient_id)
  ■     ■    ngo   → SELECT recipient WHERE user_id = me
  ■     ■            and if body.recipient_id names someone else ■■■ 403
  ■     ■■ recipient is None ■■■ 422
  ■     ■■ not recipient.is_verified ■■■ 403 (awaiting verification)
  ■     ■■ donation.recipient_id = recipient.id
  ■     ■■ recipient.accepted_donations += 1
  ■     ■■ donation.match_score = rank_recipients(...)[0].overall_score
  ■              (freeze the score the decision was made on)
  ■
  ■■ _record(): INSERT status_events
  ■     (from_status=<old>, to_status=ACCEPTED, actor_id=me,
  ■      occurred_at=SERVER clock — never from the client)
  ■     donation.status = ACCEPTED
  ■
  ■■ db.commit()   ← one transaction: event + status + counter, all or nothing
  ■
  ▼
donation_out(reloaded donation)  → DonationOut (camelCase JSON)
  ■
  ▼■■■■■■■■■■■■ back over the network ■■■■■■■■■■■■■
  ■
  ▼
AppContext re-reads the affected slice from the server (NOT an optimistic patch)
  ▼
React re-renders; useAction shows a success toast; spinner clears
```

2.6 Data flow — where each number comes from

This is worth internalising, because it is the project's cleanest idea:



Nothing in that chain is self-reported by a user. That is the point.

3. Folder & File Structure

3.1 Repository layout

```
UCS503P-202627-FoodBridge-AI/
■■■ .github/workflows/mkdocs.yml    ONLY workflow — docs, NOT tests
■■■ .claude/launch.json             dev-server launch config (frontend only)
■■■ .venv/                          Python virtualenv (not committed)
■■■ assets/                         docs theme assets (template)
■■■ code/                           ■■■ THE BACKEND
■   ■■■ foodlink/                   the actual application package
■   ■■■ tests/                      37 pytest tests
■   ■■■ requirements.txt            runtime deps
■   ■■■ requirements-dev.txt        + pytest, httpx
■   ■■■ foodlink.db                 SQLite file (untracked, correctly)
■   ■■■ Makefile                     STALE C++ template leftover
■   ■■■ src/ , inc/ , run_main.o     STALE C++ template leftovers
■■■ frontend/                       ■■■ THE FRONTEND
■   ■■■ src/                        83 .ts/.tsx files
■   ■■■ package.json
■   ■■■ vite.config.ts              the /api proxy lives here
■   ■■■ tailwind.config.js          the retuned palette
■■■ docs/                           mkdocs markdown
■■■ journals/                       per-member course journals
■■■ project-proposal/               LaTeX report
■■■ project-report-prototype-stage/
■■■ project-report-final/
■■■ foodlink.db                     a second SQLite file at root
■■■ mkdocs.yml
■■■ pyproject.toml                   still names the template project
```

Two things to be ready to explain: there are two `foodlink.db` files (root and `code/`) because the SQLite path is relative to the working directory — run uvicorn from `code/` and you get one, from the root and you get the other. And `pyproject.toml` still carries the template's name, author, and `requires-python = ">=3.8"`, which is inaccurate given the 3.10+ syntax the code actually uses.

3.2 Backend files — what to know, and how deeply

File	What it does	Depends on	Depended on by	Depth needed
`foodlink/main.py`	Creates the `FastAPI` app, adds CORS, mounts five routers, defines `/api/health`. Its `lifespan` calls `Base.metadata.create_all` — **this is the substitute for migrations**.	config, database, all routers	uvicorn, tests	**DEEP** — small file, but the `create_all` line is a design decision you must defend
`foodlink/models.py`	All six tables, `UserRole`, `DonationStatus`, `ALLOWED_TRANSITIONS`, `SELF_SIGNUP_ROLES`, the `UtcDateTime` type, and the `reliability_score` property.	database	everything	**DEEP** — this is the most important file in the project
`foodlink/security.py`	`hash_password`, `verify_password`, `create_access_token`, `get_current_user`, `require_roles`. 74 lines.	config, database, models	every router	**DEEP** — memorise it; it is short and it is where auth interview questions land
`foodlink/matching.py`	Haversine distance, five scoring functions, `WEIGHTS`, `score_pair`, `rank_recipients`. Pure functions — no database access at all.	models (types only)	donations router, serialize	**DEEP** — the "AI" of the project; expect to derive a score by hand

`foodlink/routers/donations.py`	The donation lifecycle: create, list, get, matches, status transition. `TRANSITION_ROLES` lives here.	everything	app	**DEEP** — `update_status` is the single most important function
`foodlink/routers/auth.py`	register, login, me, profile update, password change.	security, models, schemas	app	**DEEP**
`foodlink/routers/admin_users.py`	User CRUD, verify/revoke organisations, expiry sweep. Router-level role gate.	security, models	app	**DEEP** — the last-admin guard is a favourite question
`foodlink/schemas.py`	Every request/response shape. `to_camel` alias generator. Field constraints.	models (enums)	all routers	**MEDIUM** — know the pattern and the constraints, not every field
`foodlink/config.py`	`Settings` read from env, cached with `lru_cache`.	—	database, security, donations	**MEDIUM** — know every variable and its default
`foodlink/database.py`	Engine, `SessionLocal`, `Base`, the `get_db` dependency. 34 lines.	config	everything	**MEDIUM** — understand why `get_db` is a generator
`foodlink/serialize.py`	`donation_out`: flattens a `Donation` + relations into `DonationOut`, computing `distanceKm` live.	matching, schemas	donations router	**MEDIUM**
`foodlink/routers/metrics.py`	Derives all six timing metrics from `status_events`.	models, schemas	app	**MEDIUM** — know what each metric measures
`foodlink/routers/organizations.py`	Recipients, requirements, volunteers; the `/me` endpoints.	security, models	app	**MEDIUM**
`foodlink/cli.py`	`create-admin`, `promote`, `reset-password`, `list-admins`. The bootstrap answer.	database, models, security	operators, tests	**MEDIUM** — know *why* it exists
`foodlink/seed.py`	Demo data with deadlines relative to run time.	models	demos	**CONCEPTUAL**
`tests/conftest.py`	In-memory SQLite per test via `StaticPool`, `get_db` override, helper factories.	app	all tests	**DEEP** — the `StaticPool` trick is a great question

3.3 Frontend files — what to know

File	What it does	Depth needed
`src/lib/api.ts`	**The only place the app calls `fetch`.** Token attachment, `ApiError`/`NetworkError`, the 401 handler, and every wire type mirroring Pydantic. 449 lines.	**DEEP** — the highest-value frontend file

`src/context/AuthContext.tsx`	Identity. Boot-time token→user exchange, sign in/up/out, global 401 handling with a re-entrancy guard.	**DEEP**
`src/context/AppContext.tsx`	Domain state + write-then-refetch mutations + toasts.	**DEEP**
`src/components/ProtectedRoute.tsx`	50 lines. Loading splash, redirect-to-login with `from` memory, wrong-portal redirect.	**DEEP** — small and certain to be asked
`src/lib/hooks.ts`	`useAction` (keyed in-flight tracking + toasts) and `useMatchAnalysis`.	**DEEP** — `useAction`'s key mechanism is a strong talking point
`src/lib/adapters.ts`	Wire type → app type translation. The seam that stops backend shapes leaking into 40 components.	**MEDIUM**
`src/types/index.ts`	The app's own domain types.	**MEDIUM**
`src/App.tsx`	The whole route table in one readable file.	**MEDIUM**
`src/mobile/*` (26 files)	Mobile-specific screens mounted at `/m/*`, with their own inner `MobileRole` guard.	**CONCEPTUAL** — know *that* it exists and why
`src/pages/**` (29 files)	The portal screens.	**CONCEPTUAL** — pick two you can walk through
`vite.config.ts`	The `/api` → `:8000` proxy. Explains why dev has no CORS.	**DEEP** — tiny file, very common question
`tailwind.config.js`	The retuned warm palette.	**CONCEPTUAL**

---

4. Frontend Deep Dive

4.1 Framework and component model

React 18.3 with TypeScript, function components only, no class components anywhere. Vite builds it; `npm run build` runs `tsc && vite build`, so **type errors fail the build** — that is the closest thing this project has to a frontend test gate.

4.2 The two-context state model

There is no Redux, no Zustand, no React Query. State lives in two Contexts.

`AuthProvider` — identity only:

```
{ user, isLoading, expiredMessage, clearExpiredMessage,
 signIn, signUp, signOut, updateProfile, changePassword }
```

`AppProvider` — domain data and mutations:

```
{ state: { donations, requirements, recipients, volunteers,
          activity, stats, toasts, isLoading, loadError },
 refresh, createDonation, createRequirement, updateDonationStatus,
 setRecipientVerified, setAvailability, showToast, dismissToast }
```

The write strategy — know this cold

From the file's own header comment (AppContext.tsx:1):

> *Writes go to the server and the affected slice is re-read from it. That is slower than patching local state optimistically, but it is the only way the client and server cannot disagree.*

This is write-then-refetch, not optimistic update. The tradeoff is deliberate and you must be able to defend it: the server owns the lifecycle rules (`ALLOWED_TRANSITIONS`), the role checks, and the server-stamped history. If the client patched state optimistically and the server rejected the transition with a 409, the UI would have to unwind — and any bug in that unwind shows the user a state that

never existed. Re-reading costs a round trip and buys the guarantee that the screen shows what the database holds.

When would optimistic updates be right instead? When the action almost never fails, latency is user-visible, and a wrong guess is cheap to correct — a "like" button, a checkbox. Not a food-custody transition.

4.3 Authentication state — the boot sequence

```
App mounts
  ■
  ■■ isLoading initialised to  getToken() !== null
  ■     (if there is no token, we already know nobody is signed in —
  ■      no splash flash for anonymous visitors)
  ■
  ■■ if no token → isLoading=false, render immediately
  ■
  ■■ if token exists:
       GET /api/auth/me   with the stored token
         ■■ 200 → setUser(toUser(apiUser))
         ■■ error → setUser(null)
                    (a 401 already cleared the token in api.ts;
                     a network error leaves it so a retry can work)
       finally → isLoading = false
```

The key decision: only the *token* is persisted. The user object is re-fetched from `/api/auth/me` on every boot rather than restored from localStorage. If it were cached locally, a suspended or re-roled account could keep acting on a stale copy of its own permissions until it happened to refresh.

The global 401 handler and its re-entrancy guard

`api.ts` holds a module-level `onUnauthorized` callback that `AuthContext` registers once. Any 401 from anywhere in the app clears the token and fires it.

The subtle part is the guard (AuthContext.tsx:56):

```
const expiring = useRef(false);
// ...
if (expiring.current) return;
expiring.current = true;
// ... announce expiry once ...
window.setTimeout(() => { expiring.current = false; }, 1000);
```

A dashboard fires five parallel requests. If the token has expired, all five come back 401. Without the guard, the user gets five "session expired" announcements. `useRef` is used rather than `useState` because the flag must be readable and writable *synchronously* within one event loop turn — a state update would not be visible to the second 401 arriving in the same tick.

4.4 Protected routes

```
if (isLoading) return <AuthSplash />;                  // 1. don't flash login
if (!user) return <Navigate to="/login" replace
                    state={{ from: location.pathname }} />;  // 2. remember
if (allow && !allow.includes(user.role))
 return <Navigate to={HOME_PATH[user.role]} replace />;      // 3. own portal
return <Outlet />;                                            // 4. render
```

Four behaviours, each earning its place:

1. **The splash exists to prevent a flash of the login screen** in front of someone who *is* signed in, while their stored token is still being exchanged.
2. `state={{ from }}` lets login resume where they were headed.
3. Wrong portal sends them to their own home rather than to an error.
4. `<Outlet/>` renders the matched child route — this is why the guard wraps route *groups* instead of being repeated in every page.

The sentence that must accompany this in an interview (it is in the file's own comment): *this guard is a UX affordance, not a security control.* It runs in the browser, where the user controls everything. The server checks the role on every single request regardless. Deleting `ProtectedRoute` entirely would make the app ugly and full of 403s — it would not grant anyone any data.

4.5 The API layer

`lib/api.ts` centralises four concerns so they are solved once instead of forty times:

1. **Token attachment** — `getToken()` from localStorage into an `Authorization: Bearer` header, unless `anonymous: true`.
2. **Error normalisation** — any non-ok response becomes a thrown `ApiError` carrying the server's own `detail`, which the backend writes as a sentence meant for a person.

3. **Global 401** — clears the token and fires the handler.
4. **Wire types** — TypeScript interfaces mirroring the Pydantic schemas.

`extractDetail` — a genuinely nice piece of code

FastAPI returns `detail` two different ways: a **string** for your own `HTTPException`s, and a **list of field errors** for Pydantic validation failures. Both must become one readable sentence:

```
{"detail": "This organisation is awaiting verification..."}
  → shown as-is

{"detail":[{"loc":["body","pickupDeadline"],"msg":"Value error, ..."}]}
  → take the last element of `loc`  → "pickupDeadline"
  → humanise it                     → "Pickup deadline"
  → strip Pydantic's "Value error, " prefix
  → join multiple with ". "
```

The dead-backend case

```
if (response.status >= 500 && parsed === null) throw new NetworkError();
```

A stopped backend does not produce a failed `fetch` — the Vite proxy answers with a 500 and an HTML body. Reporting "Request failed (500)" would be true and useless. Naming it *"Cannot reach the FoodLink server. Is the backend running?"* is the difference between a debuggable app and a mysterious one.

4.6 `useAction` — keyed loading states

```
const { run, isPending } = useAction();
run('accept-42', () => updateDonationStatus('42', 'ACCEPTED'),
   { success: { message: 'Donation accepted' },
     errorTitle: 'Could not accept' });
```

The **key** is the whole point. On a list of twenty donation rows sharing one handler, a single boolean `isLoading` would spin all twenty. Tracking `pendingKey` means only the clicked row spins, while `isBusy` can still disable the rest.

It also tracks `mounted` via a ref, so a component unmounted mid-request does not call `setState` on a dead component.

4.7 Forms, validation, loading and errors

- **Forms are uncontrolled-to-controlled React state** — plain `useState` per field. **There is no form library** (no React Hook Form, no Formik).
- **Client-side validation is minimal.** The real validation is Pydantic on the server, and the error text comes back through `extractDetail`. Be honest about this: it means an invalid form costs a round trip. It also means there is exactly one source of validation truth, which is why the messages are always consistent.
- **Loading** appears in three forms: `AuthSplash` during token exchange, `state.isLoading` for the initial data load, and `useAction`'s `pendingKey` for individual actions.
- **Errors** surface as toasts carrying the server's own sentence, plus `state.loadError` for a whole-app load failure and a `DataGate` component for the empty/error/loaded branch.

4.8 The mobile branch

`frontend/src/mobile/` holds 26 files — a parallel set of screens (`DonorHome`, `NGOAvailable`, `VolunteerTasks`, `CreateDonationCamera`, …) mounted through `MobileApp.tsx`.

Be precise about how it is reached: it is a **separate URL space**, not a viewport branch. `App.tsx` mounts it as `<Route path="/m/*" element={<MobileApp/>} />` behind a plain `ProtectedRoute` (any signed-in account). `MobileApp` then runs its own nested router with an inner `MobileRole` guard per portal, redirecting to `MOBILE_HOME[role]`. So a user is on the mobile UI because the **URL** says `/m/...`, not because their screen is narrow.

`useIsMobile.ts` exists but is imported nowhere — verified by grep. It is dead code: a `matchMedia` hook for a 768px breakpoint that nothing calls. Do not claim the app auto-switches by viewport; it does not.

This is **not** a separate application and not React Native — it is the same SPA with a second component tree, so phone layouts can be genuinely touch-first (bottom nav in `nav.ts`, a camera-style capture flow) rather than a desktop grid squeezed narrow. Two costs worth naming: a second set of screens to keep in sync, and the fact that nothing routes users to `/m/` automatically, so a phone visitor lands on the desktop layout unless they know the URL.

5. Backend Deep Dive

5.1 Server composition

`main.py` is deliberately thin: build the app, add CORS, mount five routers, expose `/api/health`. The one piece of real behaviour is the lifespan hook:

```
@asynccontextmanager
async def lifespan(_: FastAPI):
   Base.metadata.create_all(bind=engine)   #  not a migration system
   yield
```

`create_all` issues `CREATE TABLE IF NOT EXISTS` for every model. It creates tables that are missing. It **does not alter tables that already exist.** Add a column to a model, restart, and the column silently will not appear — every query touching it then fails. The code's own comment concedes this: *"Introduce Alembic before the schema has to change without dropping data."* Today the answer is "delete `foodlink.db` and start over", which is acceptable for coursework and unacceptable in production.

5.2 Dependency injection — the backbone

FastAPI's `Depends` is what keeps the routers this short. Three dependencies carry the whole system:

```
def get_db() -> Iterator[Session]:      # database.py
   db = SessionLocal()
   try:    yield db
   finally: db.close()                # runs even if the handler raises
```

A generator dependency: everything before `yield` is setup, everything after is teardown. FastAPI runs the teardown when the response is finished, so a session can never leak — including on an exception path.

```
def get_current_user(token=Depends(oauth2_scheme), db=Depends(get_db)) -> User
```

Dependencies compose — this one depends on two others, and FastAPI resolves the graph and caches each dependency per request (so `get_db` runs once even though several dependencies ask for it).

```
def require_roles(*roles: UserRole):    # a dependency FACTORY
   def dependency(user: User = Depends(get_current_user)) -> User:
       if user.role not in roles: raise HTTPException(403, ...)
       return user
   return dependency
```

This is a **closure**: `require_roles(UserRole.ngo)` returns a *new* function that has captured `roles`. That is what allows the parameterised, declarative usage at every endpoint — and the router-wide form:

```
router = APIRouter(prefix="/api/admin",
                   dependencies=[Depends(require_roles(UserRole.admin))])
```

Every path under `/api/admin` is gated by one line, so a new admin endpoint **cannot** be added unprotected by forgetting a decorator. That is defence by construction, and it is a strong thing to point at.

5.3 Validation — three distinct layers

Layer	Where	Example	Failure
Schema	Pydantic in `schemas.py`	`password: str = Field(min_length=8)`; `latitude: float = Field(ge=-90, le=90)`; `quantity: int = Field(gt=0)`; `role` must be in `SELF_SIGNUP_ROLES`	**422** with a field-level list
Business	Inside the path function	deadline must be in the future; recipient must be verified; email not already taken	**422 / 403 / 409** with a sentence
Lifecycle	`ALLOWED_TRANSITIONS` + `TRANSITION_ROLES`	can't go `DELIVERED` → `AVAILABLE`; a donor can't set `ACCEPTED`	**409 / 403**

Note the deliberate choice at schemas.py:45: the "no self-made admins" rule is a **schema validator, not a router check**, so it appears in the published OpenAPI document. The API's contract itself says `admin` is not an accepted value.

5.4 Error handling

There is **no custom exception handler and no global error middleware**. Every error is a `raise HTTPException(status_code=..., detail=...)`, which FastAPI renders as `{"detail": ...}`. The `detail` strings are written as sentences for humans — *"This is the last active administrator — appoint another one first"* — and `lib/api.ts` passes them straight to a toast. That is a coherent, deliberate pipeline: **one message, written once, on the server.**

The gap: an *unhandled* exception (a genuine bug) returns FastAPI's default 500 with no body. `api.ts` translates that into `NetworkError`, so the user is told "cannot reach the server" when the server is actually up but broken. Slightly misleading, and worth naming as a known rough edge.

5.5 Every API endpoint in detail

`GET /api/health`

- **Purpose:** liveness check.
- **Auth:** none. **Authz:** none.
- **Response:** `200 {"status":"ok","time":"<ISO8601 UTC>"}`
- **Note:** does not touch the database, so it proves the process is up, not that the database is reachable.

`POST /api/auth/register`

- **Purpose:** create a donor/ngo/volunteer account and sign them straight in.
- **Auth:** none. **Authz:** role must be in `SELF_SIGNUP_ROLES`.
- **Input:** JSON `RegisterRequest` — `name`, `email`, `password`, `role`, and optional `organization`, `phone`, plus NGO-only `organizationType`, `location`, `latitude`, `longitude`, `capacity`.
- **Validation:** valid `EmailStr`; password 8–128; name 1–120; lat/long in range; `capacity > 0`; **`role` rejected if `admin`**.
- **Business logic:** lowercase the email → reject duplicates → bcrypt the password → `INSERT users` → `flush()` to obtain the id → if `volunteer`, insert a `Volunteer` row; if `ngo`, insert a `Recipient` row with `is_verified=False` → commit → mint a token.
- **DB ops:** 1 `SELECT`, 1–2 `INSERT`, 1 commit.
- **Response:** `201 { accessToken, tokenType, user }`
- **Errors:** `409` duplicate email; `422` validation, including the admin rejection.
- **Why the side-effect rows matter:** without them an NGO account would be authenticated but unable to accept anything, and a volunteer could not be assigned a task. Registration creates the *profile*, not just the login.

`POST /api/auth/login`

- **Purpose:** exchange credentials for a token.
- **Auth:** none.
- **Input: form-encoded** (not JSON) — `username` (the email) and `password`, per `OAuth2PasswordRequestForm`.
- **Business logic:** look up by lowercased email → `bcrypt.checkpw` → **`is_active` check** → mint token.
- **Response:** `200 { accessToken, tokenType, user }`
- **Errors:** `401` "Incorrect email or password" — **the same message whether the email is unknown or the password is wrong**, so the endpoint cannot be used to enumerate which addresses have accounts. `403` for a suspended account, with a message that actually explains the situation.
- **Interview note:** be ready for "why is login form-encoded when everything else is JSON?" Answer: it implements the OAuth2 password-flow shape that FastAPI's `OAuth2PasswordRequestForm` and the Swagger "Authorize" button expect, which is why `python-multipart` is a dependency.

`GET /api/auth/me` · **`PATCH /api/auth/me`** · **`POST /api/auth/password`**

- **Auth:** required. **Authz:** any signed-in role; acts only on yourself.

- `PATCH` accepts `ProfileUpdate` — **`name`, `organization`, `phone` only.** `role`, `email`, and `is_active` are deliberately absent: they decide what the account may do and who it is, so they belong to an administrator. This is privilege-escalation prevention by schema design.
- `POST /password` requires `currentPassword` **and** a `newPassword` of 8–128, so a stolen but unlocked session cannot silently change the password.

`POST /api/donations`

- **Purpose:** post surplus food.
- **Auth:** required. **Authz:** `donor` or `admin`.
- **Input:** `DonationCreate` — food name, category, quantity, unit, storage type, description, location, **latitude, longitude**, optional `preparedAt`, and `pickupDeadline`.
- **Validation:** `quantity > 0`; lat/long in range; `foodName` 1–160; **`pickupDeadline` must be in the future** (naive datetimes are read as UTC).
- **Business logic:** insert the donation as `AVAILABLE` → append an `AVAILABLE` status event → **immediately rank all recipients** → if any qualify, store the top score and transition to `MATCHED` with a note naming the top organisation → commit.
- **DB ops:** 1 `INSERT` donation, 1 `SELECT` all recipients, 1–2 `INSERT` events, 1 commit.
- **Response:** `201 DonationOut`
- **Errors:** `422` past deadline or schema failure; `403` wrong role.
- **Critical nuance:** `MATCHED` **assigns nothing.** `recipient_id` is still null. It records "the system has a suggestion". Only `ACCEPTED` binds a recipient. Interviewers love this distinction.

`GET /api/donations`

- **Auth:** required. **Authz:** any signed-in role.
- **Query:** `status` (repeatable), `mine` (bool, default **false**), `limit` (default 100, max 500).
- **Business logic:** when `mine=true`, scope by role — donor by `donor_id`, ngo by their recipient id, volunteer by their volunteer id. The `-1` fallback for an account with no profile row is a neat trick: it matches nothing rather than erroring.
- **Ordered by `pickup_deadline`** ascending — most urgent first, which is the right default for this domain.
- **Uses `selectinload`** for donor, recipient, volunteer→user, and events, which is what stops the N+1 problem (§16).
- **Security note:** with `mine` defaulting to false, **any authenticated user can list every donation on the platform**, including exact coordinates and donor names. See §8.

`GET /api/donations/{id}`

- **Auth:** required. **Authz: none beyond being signed in** — any authenticated user can read any donation by id.

`GET /api/donations/{id}/matches`

- **Purpose:** the ranked recipient list with per-criterion reasoning.
- **Auth:** required. **Authz:** any signed-in role. **Query:** `limit` ≤ 25.
- **Business logic:** load all recipients, run `rank_recipients`, return `MatchOut` objects carrying the overall score, distance, all five sub-scores, and human-readable `reasons`.
- **This endpoint is the "explainable AI" claim.** The score is never a bare number — it always arrives with its components and its reasons.

`POST /api/donations/{id}/status` ← **the most important endpoint**

- **Purpose:** drive the lifecycle.
- **Auth:** required. **Authz:** layered — see below.
- **Input:** `StatusUpdate` — `status`, optional `recipientId`, optional `note`.

- **Logic, in order:** 1. `404` if the donation does not exist. 2. **`409`** if the target is not in `ALLOWED_TRANSITIONS[current]`. 3. **`403`** if the caller's role is not in `TRANSITION_ROLES[target]` — *except* that the owning donor may always `CANCELLED`. 4. Target-specific side effects (recipient resolution and verification check; courier claim guard; completion counters). 5. Append a `StatusEvent`, set the status, commit — **one transaction**.
- **Errors:** `404`, `409` illegal transition, `409` pickup already claimed, `403` wrong role, `403` accepting for another organisation, `403` unverified organisation, `422` no recipient/courier profile resolved.

`GET /api/recipients` · **`GET|PATCH /api/recipients/me`**

- `GET /api/recipients` — **any signed-in role**; returns every organisation including `contactPerson` and `phone`. See §8.
- `/me` — `ngo` only. `PATCH` is how an organisation supplies the coordinates that make it matchable at all. **`is_verified` is not in `RecipientUpdate`** — an organisation cannot vouch for itself.

`GET|POST /api/requirements`

- `GET` — any signed-in role; active requirements, newest first.
- `POST` — `ngo` or `admin`; always attached to *your own* organisation via `_own_recipient`, never to an id from the request body.

`GET /api/volunteers` · **`GET|PATCH /api/volunteers/me`**

- `GET /api/volunteers` — **`admin` or `ngo` only.** The code comments why: *"it is a list of people's phone numbers."* Couriers cannot read the roster.
- `/me` — `volunteer` only. `VolunteerUpdate` permits `isAvailable`, `location`, `latitude`, `longitude` — and deliberately **not** `completedDeliveries` or `rating`, because those are earned, so they are the server's to maintain.

`GET /api/metrics`

- **Auth:** required. **Authz:** any signed-in role.
- Loads **all** donations with their events and computes seven counters plus four derived figures in Python. Full table scan into memory — the clearest scalability bottleneck (§16).

Admin endpoints (all behind the router-level `require_roles(admin)`)

Endpoint	Purpose	Notable logic
`GET /api/admin/users`	List accounts	Filters `role`, `include_i`

`POST /api/admin/users`	Create any account **including admin**	Mirrors registration's side-effect rows; an NGO created here is `is_verifi` because an admin creating it *is* the vouching
`PATCH /api/admin/users/{id}`	Suspend, restore, rename, re-role	**Two lockout guards:** you cannot suspend/de yourself, and the last active admin cannot be suspended/
`POST /api/admin/recipients/{id}/verify`	Vouch for an organisation	Unlocks acceptance and match ranking
`DELETE /api/admin/recipients/{id}/verify`	Revoke	
`POST /api/admin/maintenance/expire`	Expiry sweep	Moves overdue `AVAILABLE` donations to `EXPIRED` with an event; returns `{"expired n}`

The expiry sweep has no scheduler. No cron, no APScheduler, no Celery. Someone or something must call it. Until they do, the expiry-loss metric understates reality. This is an honest gap to name.

---

6. Database Deep Dive

6.1 Schema overview

6.2 Table-by-table

`users` — every account, all four roles in one table

Column	Type	Notes
`id`	int PK	
`name`	String(120)	
`email`	String(255) **unique, indexed**	Always lowercased before insert/looku
`password_hash`	String(255)	**bcrypt hash — never a plaintext password**
`role`	Enum(UserRole) **indexed**	donor / ngo / volunteer / admin
`organization`	String(160) nullable	Free text

`phone`	String(32) nullable	
`is_active`	Boolean, default true	**Checked on every request**, not just login
`created_at`	UtcDateTime, `server_default=func.now()`	

Design decision — single-table inheritance for roles. One `users` table with a `role` column, rather than separate `donors`/`ngos`/`volunteers` tables. Authentication is then one query against one table regardless of role, and adding a role is adding an enum value. Role-specific *data* that genuinely differs lives in the satellite tables. The cost: `organization` is meaningless for a volunteer, so the table has columns that are null for most rows.

Two Python-level properties, not columns: `initials` (derived from the name for avatars) and `recipient_id`/`volunteer_id` (the profile this account acts for). Computing them avoids storing data that can go stale.

`recipients` — organisations that can receive food

Key fields: `is_verified` (the trust gate), `capacity` (feeds two scoring terms), nullable `latitude`/`longitude`, and the two counters `accepted_donations` / `completed_donations`.

Why lat/long are nullable: an NGO signs up before pinning its address. A recipient without coordinates simply is not matchable (`score_pair` returns `None`), which is correct behaviour rather than an error — it keeps registration one short step instead of a survey.

`reliability_score` is a computed property, not a column:

```
@property
def reliability_score(self) -> int:
   if self.accepted_donations < 3:
       return 85               # optimistic prior for newcomers
   return round(100 * self.completed_donations / self.accepted_donations)
```

Two things to be able to defend. First, it is **derived**, so it can never disagree with the counters it comes from. Second, the **85 for organisations with fewer than three accepted donations** is a cold-start prior: an organisation with one lucky completion would otherwise sit at 100% and permanently outrank everyone, and a brand-new kitchen at 0% would never be matched and so could never earn a record. It is a deliberate hedge, and `3` is a magic number chosen by judgement, not by data — say so.

`volunteers` — courier profiles

`user_id` is **unique** (a strict 1:1 with `users`), plus availability, location, `completed_deliveries` and `rating` (default 5.0). **`rating` is never written by any API path** — no endpoint updates it, and `VolunteerUpdate` deliberately excludes it, so every courier created through the app sits at 5.0 forever. Only `seed.py` sets non-default values, for demo realism. It is a schema placeholder for an unimplemented feature: **do not claim the platform rates couriers.**

`donations` — the central entity

Note three modelling decisions the file itself calls out:

1. **Coordinates are stored; `distanceKm` is not.** Distance is a relationship between two places, not a property of a donation, and the matcher needs it against many recipients. `serialize.donation_out` computes it on the fly.
2. **`pickup_deadline` is an absolute instant, not a display string** like "8:00 PM". All the urgency and overdue logic depends on comparing to real time.
3. **`match_score` is frozen at the moment of matching**, so the number shown later is the one the decision was actually made on — not a number that silently drifts as capacity and reliability change.

`status_events` — the append-only ledger

Column	Purpose
`donation_id` FK, indexed	Which donation
`from_status` nullable	Null for the very first (creation) event

`to_status` indexed	The new state
`actor_id` FK nullable	Who did it — null for system actions like the expiry sweep
`note`	e.g. "Top match: Sewa Kitchen"
`occurred_at` indexed	**Stamped by the server. Never accepted from a client.**

This is the best design decision in the project. The alternative — a `matched_at`, `accepted_at`, `picked_up_at`, `delivered_at`, `completed_at` column on `donations` — is what most student projects do. Why this is better:

- **Metrics become queries over evidence** rather than trust in columns.
- **The audit trail includes who acted**, which columns cannot express.
- **Adding a state costs nothing.** New timestamp columns would mean a migration each time.
- **Re-entering a state is representable.** `MATCHED → AVAILABLE → MATCHED` is legal in `ALLOWED_TRANSITIONS`; a single `matched_at` column would be overwritten and the first attempt lost.

The tradeoff to concede: reading "when was this accepted?" is a scan of the donation's events (`Donation.timestamp_of`) instead of a column read. At this scale, irrelevant; at large scale, this is exactly the kind of thing you would denormalise or materialise.

`requirements` — standing needs

Lets demand be visible before supply exists. `is_active` allows soft-deletion, and `daily_recurring` marks a standing need. **There is no `PATCH` or `DELETE` for requirements** — they can be created and listed, but never edited or deactivated through the API. A real functional gap (§22).

6.3 Relationships

Relationship	Cardinality	Notes
`users` → `recipients`	1:1 optional	`uselist=False`; only NGO accounts
`users` → `volunteers`	1:1 optional	`user_id` unique; only volunteer accounts
`users` → `donations`	1:N	via `donor_id`
`recipients` → `donations`	1:N optional	null until `ACCEPTED`
`volunteers` → `donations`	1:N optional	null until `VOLUNTEER_ASSIGNED`
`donations` → `status_events`	1:N	`cascade="all, delete-orphan"`, ordered by `occurred_at`
`recipients` → `requirements`	1:N	`cascade="all, delete-orphan"`
`users` → `status_events`	1:N optional	as `actor`

There are no many-to-many relationships and no association tables. If asked where one *would* appear: a donation splittable across several recipients, or recipients declaring accepted food categories.

6.4 Constraints and indexes

Enforced by the database: `users.email` unique; `volunteers.user_id` unique; foreign keys on every relationship; `NOT NULL` on every non-optional column; enum columns.

Indexed: `users.email`, `users.role`, `donations.donor_id`, `donations.status`, `donations.pickup_deadline`, `donations.recipient_id`, `donations.volunteer_id`, `donations.created_at`, `status_events.donation_id`, `status_events.to_status`, `status_events.occurred_at`, `requirements.recipient_id`, `recipients.name`.

The indexes match the actual access patterns: filter by status, order by deadline, scope by owner, and fetch a donation's events.

Enforced only in application code (a real observation): - The state machine — no database CHECK constraint mirrors `ALLOWED_TRANSITIONS` - `capacity > 0`, `quantity > 0` — Pydantic only - Coordinate ranges — Pydantic only - `accepted_donations` / `completed_donations` consistency

Consequence: anything writing to this database *other than* the API can violate every one of those rules. That is the honest cost of putting invariants in the application layer, and worth conceding before an interviewer finds it.

`PRAGMA foreign_keys` is never set. SQLite does **not** enforce foreign keys by default, so on the default configuration the FK constraints are declarative only. On Postgres they would be enforced. This is a genuine finding worth mentioning.

6.5 The `UtcDateTime` type decorator — know this one

The single most subtle piece of code in the project (models.py:31).

The problem: SQLite has no timezone type. `DateTime(timezone=True)` stores the naive wall clock and hands it back with no offset. The API then serialises `2026-09-01T08:05:44` with no zone, and the browser reads it as *local* time. A deadline four hours out appears ninety minutes *past* in IST. Silently wrong, twice over.

The fix — a `TypeDecorator` wrapping `DateTime`:

```
process_bind_param   (Python → DB):  naive → assume UTC; aware → convert to UTC
process_result_value (DB → Python):  naive → attach UTC; aware → convert to UTC
```

Every datetime is timezone-aware UTC in Python, always. Postgres already behaves this way, so the decorator passes through unchanged — the code is portable across both. `cache_ok = True` lets SQLAlchemy cache compiled statements using this type.

Why this matters for an interview: it is the clearest example in the codebase of a bug that would never show up in a unit test but would corrupt every deadline in production, fixed at the type layer so no individual query has to remember.

6.6 Migrations

There are none. No Alembic, no migration directory, no versioning. Schema comes from `Base.metadata.create_all` at startup and in `cli._session()`.

What that means concretely: - A fresh clone runs with zero setup. - Adding a column to a model does not alter an existing table. The app starts fine and then fails on any query touching the new column. - There is no rollback and no schema history. - **Today's workaround:** delete the `.db` file and re-seed. - **The fix:** `pip install alembic`, `alembic init`, autogenerate an initial revision from the existing models, and replace the `create_all` call with `alembic upgrade head`.

6.7 Important queries

The loaded donation query — how N+1 is avoided:

```
select(Donation).options(
   selectinload(Donation.donor),
   selectinload(Donation.recipient),
   selectinload(Donation.volunteer).selectinload(Volunteer.user),
   selectinload(Donation.events),
)
```

Without this, rendering 100 donations would fire 1 query for the list plus 4 per row = **401 queries**. With it: **5 queries** total, regardless of row count. `selectinload` issues a second `SELECT ... WHERE id IN (...)` per relationship rather than a join, which avoids row multiplication on the one-to-many `events`.

The last-admin guard:

```
select(func.count()).select_from(User)
 .where(User.role == UserRole.admin, User.is_active.is_(True))
 .where(User.id != excluding)
```

The expiry sweep:

```
select(Donation).where(
   Donation.status.in_([AVAILABLE, MATCHED]),
   Donation.pickup_deadline < now,
)
```

Both indexed columns — this stays fast as the table grows.

---

7. Authentication & Authorization

This is the section most likely to decide a technical interview. Everything below is traced from the actual code.

7.1 The complete journey

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 1. REGISTRATION                                                            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Browser: Login.tsx form → AuthContext.signUp(body) → api.register(body)
        POST /api/auth/register   (anonymous: true — skip the 401 handler)
                  ■
Server:  Pydantic RegisterRequest validates BEFORE the function runs
        ■■ email is a real address (EmailStr)
        ■■ 8 ≤ password ≤ 128
        ■■ role ∈ {donor, ngo, volunteer}   ← admin REJECTED here (422)
                  ■
        email.lower()  →  SELECT users WHERE email = ?
        ■■ exists ■■■ 409
                  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 2. PASSWORD HANDLING                                                       ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
                  ■
        • gensalt() → a NEW random salt per password
        • the salt is stored INSIDE the hash string, so no salt column
        • default cost factor 12 → ~2^12 rounds → deliberately slow
        • the plaintext is never stored, never logged, never returned
                  ■
        INSERT users(password_hash=<hash>, ...)
        flush()   ← get the id WITHOUT committing yet
                  ■
        role=volunteer → INSERT volunteers(user_id=...)
        role=ngo       → INSERT recipients(user_id=..., is_verified=FALSE)
                  ■
        commit()  ← account + profile land together, or not at all

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 3. LOGIN & CREDENTIAL VERIFICATION                                         ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Browser: api.login(email, password)
        POST /api/auth/login   Content-Type: x-www-form-urlencoded
        username=<email>&password=<plain>
                  ■
Server:  SELECT users WHERE email = lower(username)
                  ■
        bcrypt.checkpw(plain.encode(), stored_hash.encode())
        ■■ reads the cost + salt out of the stored hash
        ■■ re-hashes the candidate with them
        ■■ compares in constant time
                  ■
        user is None OR checkpw false
             ■■■■■ 401 "Incorrect email or password"
                  (ONE message for both — no account enumeration)
                  ■
        not user.is_active ■■■ 403 "This account has been deactivated..."
                  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 4. TOKEN CREATION                                                          ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        payload = { "sub":  str(user.id),
                    "role": user.role.value,
                    "exp":  now_utc + 720 minutes }     ← 12 HOURS
        jwt.encode(payload, settings.secret_key, algorithm="HS256")
                  ■
        Structure:  base64(header).base64(payload).base64(signature)
         The payload is SIGNED, NOT ENCRYPTED — anyone can read it.
          Never put a secret in it. This one holds only an id and a role.
                  ■
        200 { accessToken, tokenType: "bearer", user: {...} }

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 5. STORAGE                                                                 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Browser: setToken(response.accessToken)
        → localStorage['foodlink.token'] = <jwt>
        → wrapped in try/catch (private mode / storage disabled)
        → the USER OBJECT is NOT persisted, only the token

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 6. AUTHENTICATED REQUEST                                                   ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        request() reads getToken() and sets
        Authorization: Bearer <jwt>     on EVERY call except anonymous ones

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 7. BACKEND VERIFICATION      [security.py:41 get_current_user]             ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        OAuth2PasswordBearer pulls the token out of the header
                  ■
        jwt.decode(token, secret_key, algorithms=["HS256"])
        ■■ signature wrong      → PyJWTError  → 401
        ■■ exp passed           → PyJWTError  → 401
        ■■ malformed            → PyJWTError  → 401
        ■■ "sub" missing/not int→ KeyError/ValueError → 401
                  ■
        user = db.get(User, user_id)      ■■■ DATABASE READ, EVERY REQUEST
        ■■ user is None        → 401   (deleted account)
        ■■ not user.is_active  → 401   (suspended MID-SESSION)
                  ■
        return the LIVE User row  (the token's `role` claim is IGNORED)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 8. AUTHORIZATION            [security.py:62 require_roles]                 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        Layer 1  role gate:     user.role ∈ allowed?      else 403
        Layer 2  ownership:     is this YOUR donation /
                                YOUR organisation?        else 403
        Layer 3  lifecycle:     is the transition legal?  else 409
        Layer 4  trust:         is the organisation
                                verified?                 else 403

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 9. LOGOUT                                                                  ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        signOut() → setToken(null); setUser(null)
         CLIENT-SIDE ONLY. No server call. No blocklist.
          A token copied before logout REMAINS VALID until `exp`.
```

7.2 Password hashing — the details

```
def hash_password(plain: str) -> str:
   return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
   try:
       return bcrypt.checkpw(plain.encode(), hashed.encode())
   except ValueError:
       return False    # malformed hash → failed login, not a 500
```

Why bcrypt rather than SHA-256: SHA-256 is designed to be *fast*, which is exactly wrong for passwords — a GPU does billions per second. bcrypt is deliberately slow and its cost factor is tunable upward as hardware improves.

Why a per-password salt matters: two users with the same password get different hashes, so a precomputed rainbow table is useless and cracking one hash tells you nothing about the other.

Why the `ValueError` catch: a corrupted or truncated hash in the database would otherwise raise and produce a 500, leaking that something is wrong with that specific account. Treating it as a failed login is both safer and honest.

bcrypt silently truncates at 72 bytes. The schema allows a 128-character password; anything beyond 72 bytes is ignored. Not a vulnerability at these lengths, but a real and quotable detail.

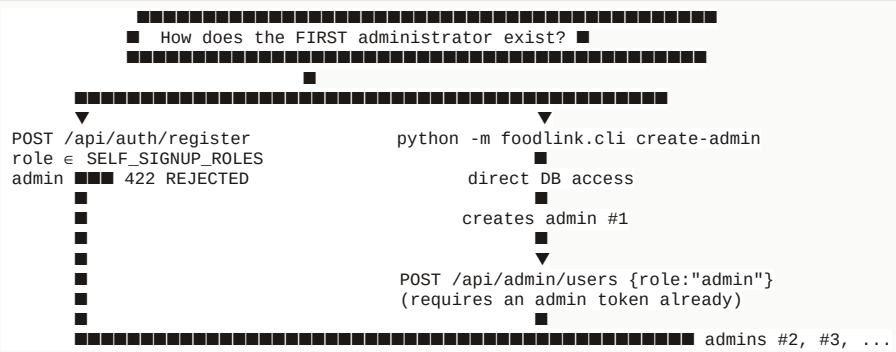
7.3 JWT specifics

Property	Value	Comment
Algorithm	**HS256**	Symmetric — one secret both signs and verifies
Claims	`sub`, `role`, `exp`	Minimal by design
Lifetime	**720 min = 12 h**	`ACCESS_TOKEN_MINUTES`, long for convenience
Secret	`FOODLINK_SECRET_KEY`	**Insecure default if unset**
Transport	`Authorization: Bearer`	Header, not a cookie
Storage	`localStorage`	Survives reload; XSS-readable
Refresh token	**none**	Verified absent
Revocation	**none**	No blocklist

HS256 vs RS256: symmetric means every service that verifies must also hold the signing key. With one backend that is fine. With several services, RS256 lets you distribute a public verification key while only the issuer holds the private one. Good answer to "would you change the algorithm?"

Why `sub` is a string: the JWT spec requires `sub` to be a string; the code does `str(user.id)` on the way in and `int(payload["sub"])` on the way out.

7.4 The two-tier admin model — a highlight



The bootstrap authority is **whoever can run commands against the database** — which is the only authority that exists before any account does. The CLI also prompts for passwords with `getpass` rather than taking them as arguments, because an argument lands in

shell history and the process list.

The lockout guards (admin.py:134):

```
demoted   = "role" in changes and changes["role"] is not UserRole.admin
suspended = changes.get("is_active") is False
losing_an_admin = user.role is admin and user.is_active and (demoted or suspended)

if losing_an_admin:
   if user.id == actor.id:                      → 409 can't do it to yourself
   if _active_admin_count(excluding=user.id)==0 → 409 last admin
```

Without these, one PATCH could leave a deployment with no administrator and no way back except shell access. Both are covered by tests (`test_the_platform_cannot_be_left_without_an_administrator`).

7.5 The four authorization layers, with examples

Layer	Mechanism	Example	Status
Role	`require_roles(...)`	A donor cannot `POST /api/admin/users`	403
Ownership	Query scoping / explicit comparison	An NGO naming another organisation's `recipientId` on accept	403
Lifecycle	`ALLOWED_TRANSITIONS`	`DELIVERED → AVAILABLE`	409
Trust	`is_verified`	An unverified kitchen accepting	403

The **role + transition matrix** — worth memorising:

Target status	Roles permitted
`MATCHED`	admin
`ACCEPTED`	ngo, admin
`VOLUNTEER_ASSIGNED`	volunteer, admin
`PICKED_UP`	volunteer, admin
`DELIVERED`	volunteer, admin
`COMPLETED`	ngo, admin
`CANCELLED`	donor (own only), admin
`EXPIRED`	admin

`COMPLETED` belongs to the NGO, not the volunteer, and that is the right call: the courier says "I delivered it", the *recipient* confirms "we received it". The party with the incentive to overstate is not the party who confirms.

7.6 Security considerations built into the auth design

Same 401 message for unknown email and wrong password `is_active` re-read every request → instant suspension Role re-read from the database, so the token's claim cannot go stale `admin` unreachable through registration, enforced in the schema Password change requires the current password `ProfileUpdate` cannot touch `role`, `email`, or `is_active` `RecipientUpdate` cannot set `is_verified` `VolunteerUpdate` cannot set `rating` or `completedDeliveries` CLI passwords prompted, never passed as arguments

---

8. Security

Everything here was checked against the code. Grep results: **no** `slowapi` or rate limiter, **no** refresh tokens, **no** raw SQL, **no** `dangerouslySetInnerHTML`, **no** `UploadFile`.

8.1 Currently protected

SQL injection — structurally prevented

Every query goes through SQLAlchemy's expression language (`select(User).where(User.email == email)`), which parameterises values. There is **no raw SQL, no string-formatted query, and no `text()` call anywhere in `code/foodlink/`** — verified by grep. Even `f"..."` never appears inside a query. This is not "we sanitise input"; it is "user data never reaches the SQL parser as code".

XSS — largely prevented

React escapes everything rendered as `{value}`. **`dangerouslySetInnerHTML` appears nowhere in the frontend** — verified by grep. There is no `eval`, no `innerHTML`, no user-supplied HTML rendering path.

Password storage

bcrypt with per-password salts and a tunable cost. No plaintext, no reversible encryption, no unsalted digest. Hashes never appear in any response schema — `UserOut` has no `password_hash` field, so it cannot leak by accident.

CSRF — not applicable by design

The token travels in an `Authorization` header, **not a cookie**. Browsers attach cookies to cross-site requests automatically; they do not attach custom headers. So the classic CSRF attack has nothing to ride on. **This is the correct answer to "how do you handle CSRF?" — not "we don't", but "the auth scheme makes it inapplicable, and here is why."** (If the project ever moved to cookie auth, CSRF protection would immediately become mandatory.)

Authorization

Four layers (§7.5), enforced server-side, with the admin router gated in one place so a new endpoint cannot be added unprotected.

Input validation

Pydantic validates types, ranges, and lengths *before* any handler runs: password 8–128, coordinates in range, `quantity > 0`, `capacity > 0`, valid email, `limit ≤ 500`, match `limit ≤ 25`.

Account enumeration

Prevented at login (one message for both failure modes). Note the **inconsistency**: registration returns a distinct 409 for a duplicate email, so registration *does* reveal whether an address has an account. That is a common and usually accepted tradeoff, but you should know it is there.

CORS

An explicit allowlist from `CORS_ORIGINS`, defaulting to the two localhost dev origins — **not** `allow_origins=["*"]`. Combined with `allow_credentials=True`, a wildcard would be rejected by browsers anyway, but the allowlist is the right shape.

Secret exposure

No `.env` file is committed, `.gitignore` covers `.env` and `.env.local`, and **no `.db` or `.env` file is tracked by git** — verified with `git ls-files`.

8.2 Potential weaknesses

CRITICAL — the default signing key

```
self.secret_key = os.getenv("FOODLINK_SECRET_KEY",
                            "dev-only-insecure-key-replace-me-in-deployment")
```

If `FOODLINK_SECRET_KEY` is not set, the app **starts anyway** with a key that is published in a public repository. Anyone could then forge a token for `{"sub":"1","role":"admin"}` and hold total control. The comment says it must be overridden; nothing enforces it. **Fix:** fail fast at startup when the environment is production, or simply refuse to boot on the default. A four-line change.

HIGH — broad read access across tenants

`GET /api/donations` defaults to `mine=false`, and `GET /api/donations/{id}` has no ownership check at all. So **any authenticated account — a volunteer, a competing NGO, a donor who signed up two minutes ago — can read every donation on the platform**, including exact latitude/longitude, donor name, and organisation. Likewise `GET /api/recipients` exposes every

organisation's `contactPerson` and `phone` to any signed-in user.

Some of this is genuinely required (a recipient must browse available donations, a courier must see pickups). But *completed*, *cancelled*, and other organisations' *accepted* donations do not need to be world-readable to every account. This is the most substantial authorization gap in the project and you should raise it yourself rather than be caught by it. **Fix:** scope the default listing by role — donors see their own, NGOs see available plus their own accepted, volunteers see assignable plus their own.

MEDIUM — no rate limiting anywhere

Verified absent. `POST /api/auth/login` can be called without limit, so password brute-forcing is bounded only by bcrypt's cost. There is no account lockout and no CAPTCHA. Every other endpoint is equally unbounded. **Fix:** `slowapi` on the auth endpoints first.

MEDIUM — token lifetime and no revocation

12-hour access tokens, no refresh token, no blocklist. Logout is client-side only, so a token captured before logout works until it expires. **Partial mitigation already present:** because `get_current_user` re-reads `is_active`, an admin *can* kill a session immediately by suspending the account. There is no equivalent for "log this device out". **Fix:** short access tokens (15 min) + refresh tokens, or a `token_version` column on `users` included in the token and compared on each request.

MEDIUM — token in localStorage

Readable by any script running on the page, so an XSS bug becomes account takeover. The code documents the tradeoff honestly: the alternative (`httpOnly` cookie) resists XSS exfiltration but introduces CSRF and complicates the SPA. Given React's escaping and no `dangerouslySetInnerHTML`, the XSS surface is currently small — but "small" is not "none".

MEDIUM — SQLite foreign keys not enforced

`PRAGMA foreign_keys = ON` is never issued, so on the default SQLite configuration the declared FKs do **not** constrain anything. Orphaned rows are possible if anything writes outside the ORM. **Fix:** a `connect` event listener issuing the pragma, or move to Postgres.

LOW — `image_url` is an unvalidated text column

`DonationCreate.image_url` is `str | None` with **no length limit and no format validation**, stored in a `Text` column. The frontend passes base64 data URLs, so a large image inflates every row and every response. A `javascript:` URL could also be stored — harmless today because nothing renders it as an `href`, but it is unvalidated data in a field named "url". **Fix:** validate the scheme, cap the length, and move real uploads to object storage.

LOW — no HTTPS enforcement

No HSTS, no redirect middleware, no `secure` flag anywhere. Over plain HTTP the bearer token is readable in transit. In practice this is a deployment concern (terminate TLS at a proxy), but nothing in the app insists on it.

LOW — no security headers

No CSP, `X-Frame-Options`, `X-Content-Type-Options`, or `Referrer-Policy`. A CSP in particular would meaningfully reduce the localStorage-token risk.

LOW — dependency vulnerabilities unmonitored

Requirements use `>=` lower bounds with no lockfile on the Python side, so two installs can resolve to different versions. There is no Dependabot, no `pip-audit`, no `npm audit` in CI (there is no test CI at all). The frontend has a `package-lock.json`; the backend has no equivalent pin.

LOW — no email verification

Anyone can register with an address they do not control. Organisation verification is a human admin step, which covers the important case, but account ownership is unverified.

8.3 Not applicable

- **File upload risks** — there is no upload endpoint. No `UploadFile`, no multipart file handling, no filesystem writes from user input. `image_url` is a string field, so the whole class of path-traversal and malicious-file problems does not arise.
- **SSRF** — the backend makes no outbound HTTP requests at all.
- **Deserialisation attacks** — no `pickle`, no `yaml.load`, no `eval`.

8.4 Recommended improvements, prioritised

#	Priority	Change	Effort
1	Critical	Refuse to start on the default `secret_key` outside development	~4 lines
2	High	Scope `GET /api/donations` and `/{id}` by role and ownership	~30 lines
3	Medium	Rate-limit the auth endpoints (`slowapi`)	~15 lines
4	Medium	Shorten access tokens; add refresh or `token_version`	~60 lines
5	Medium	`PRAGMA foreign_keys=ON` for SQLite	~6 lines
6	Low	Validate and cap `image_url`	~5 lines
7	Low	Security headers + CSP	~20 lines
8	Low	`pip-audit` / `npm audit` in CI (once CI exists)	config

---

9. API Documentation

Base URL: `http://127.0.0.1:8000` in development. Interactive docs (free from FastAPI): **`/docs`** (Swagger) and **`/redoc`**. All request and response bodies are **camelCase**. All authenticated endpoints take `Authorization: Bearer <token>`.

Authentication

`POST /api/auth/register`

Create an account and receive a token. **No auth required.**

```
{ "name": "Sharma Caterers", "email": "ops@sharma.example",
 "password": "correcthorsebattery", "role": "donor",
 "organization": "Sharma Caterers", "phone": "+91-98765-43210" }
```

NGO registration additionally accepts `organizationType`, `location`, `latitude`, `longitude`, `capacity`.

`201`

```
{ "accessToken": "eyJhbGciOiJIUzI1NiIs...", "tokenType": "bearer",
 "user": { "id": 7, "name": "Sharma Caterers", "email": "ops@sharma.example",
           "role": "donor", "organization": "Sharma Caterers",
           "initials": "SC", "recipientId": null, "volunteerId": null } }
```

Errors: `409` email taken · `422` validation, incl. `role: "admin"`

`POST /api/auth/login`

Form-encoded, not JSON. No auth required.

```
curl -X POST http://127.0.0.1:8000/api/auth/login \
 -d "username=ops@sharma.example&password=correcthorsebattery"
```

`200` same shape as register. **Errors:** `401` "Incorrect email or password" · `403` deactivated

---

`GET /api/auth/me` — the signed-in account. `200 UserOut` · `401`

`PATCH /api/auth/me` — `{name?, organization?, phone?}` only. `200` · `401`

`POST /api/auth/password` — `{currentPassword, newPassword}`. `200` · `401` wrong current · `422` too short

Donations

`POST /api/donations`

Auth: required. **Role:** `donor` | `admin`.

```
{ "foodName": "Vegetable Biryani", "category": "Vegetarian",
 "quantity": 120, "unit": "Meals", "storageType": "Room Temperature",
 "description": "Prepared for a cancelled function",
 "location": "Rajpura Road, Patiala",
 "latitude": 30.3398, "longitude": 76.3869,
 "preparedAt": "2026-09-01T11:30:00Z",
 "pickupDeadline": "2026-09-01T18:00:00Z" }
```

`201` (note it may already be `MATCHED` with a score)

```
{ "id": 42, "donorId": 7, "donorName": "Sharma Caterers",
 "foodName": "Vegetable Biryani", "quantity": 120, "unit": "Meals",
 "status": "MATCHED", "recipientId": null, "matchScore": 87,
 "distanceKm": null, "pickupDeadline": "2026-09-01T18:00:00+00:00",
 "createdAt": "2026-09-01T12:04:11+00:00",
 "events": [ {"toStatus":"AVAILABLE","fromStatus":null,
              "occurredAt":"2026-09-01T12:04:11+00:00","note":null},
             {"toStatus":"MATCHED","fromStatus":"AVAILABLE",
              "occurredAt":"2026-09-01T12:04:11+00:00",
              "note":"Top match: Sewa Community Kitchen"} ] }
```

Errors: `403` wrong role · `422` past deadline / validation

`GET /api/donations`

Auth: required. **Role:** any. **Query:** `status` (repeatable) · `mine` (default `false`) · `limit` (≤500)

```
GET /api/donations?status=AVAILABLE&status=MATCHED&limit=50
GET /api/donations?mine=true
```

`200` array of `DonationOut`, ordered by `pickupDeadline` ascending.

`GET /api/donations/{id}` — `200 DonationOut` · `404`

`GET /api/donations/{id}/matches?limit=5`

`200`

```
[ { "recipientId": 3, "recipientName": "Sewa Community Kitchen",
   "overallScore": 87, "distanceKm": 1.24,
   "distanceScore": 85, "quantityScore": 88, "capacityScore": 76,
   "deadlineScore": 100, "reliabilityScore": 92,
   "reasons": [ "1.2 km away — well inside the collection radius",
                "120 meals fits the 150-meal daily capacity closely",
                "Comfortable margin before the pickup deadline",
                "92% completion record on accepted donations" ] } ]
```

`POST /api/donations/{id}/status`

Auth: required. **Role:** depends on the target (§7.5).

```
{ "status": "ACCEPTED" }
{ "status": "ACCEPTED", "recipientId": 3 }   // admin only
{ "status": "CANCELLED", "note": "Collected internally" }
```

`200` the updated `DonationOut` with a new event appended. **Errors:** | Code | Meaning | |---|---| | `404` | Donation not found | | `409` | `Cannot move a donation from X to Y` | | `409` | `Another courier has already claimed this pickup` | | `403` | `Your role cannot set a donation to Y` | | `403` | `You can only accept a donation on behalf of your own organisation` | | `403` | `This organisation is awaiting verification and cannot accept donations yet` | | `422` | No recipient / courier profile resolved |

Organisations, requirements, couriers

Endpoint	Auth	Role	Notes
`GET /api/recipients`	✔	any	exposes contact person + phone
`GET /api/recipients/me`	✔	ngo	`422` if not linked
`PATCH /api/recipients/me`	✔	ngo	Cannot set `isVerifie`
`GET /api/requirements`	✔	any	Active only, newest first
`POST /api/requirements`	✔	ngo, admin	Always your own organisation
`GET /api/volunteers`	✔	**admin, ngo**	Closed to couriers
`GET /api/volunteers/me`	✔	volunteer	
`PATCH /api/volunteers/me`	✔	volunteer	`isAvailab` `location`, lat/long only

Metrics

`GET /api/metrics` — auth required, any role

```
{ "totalDonations": 128, "totalMeals": 9840, "completedDonations": 96,
 "activeDonations": 18, "expiredDonations": 9,
 "totalOrganizations": 12, "totalVolunteers": 21,
 "medianTimeToClaimMinutes": 23.5, "medianHandoverMinutes": 68.0,
 "rescueRatePercent": 88.6, "expiryLossRatePercent": 8.6 }
```

Any of the four derived figures is `null` when there is not enough history.

Admin — all require an admin token

Endpoint	Purpose	Notable errors
`GET /api/admin/users?role=&include_inactive=`	List accounts	`403`
`POST /api/admin/users`	Create any account incl. admin	`409` duplicate
`PATCH /api/admin/users/{id}`	Suspend / restore / rename / re-role	`409` self-demotion · `409` last admin · `404`
`POST /api/admin/recipients/{id}/verify`	Vouch	`404`

`DELETE /api/admin/recipients/{id}/verify`	Revoke	`404`
`POST /api/admin/maintenance/expire`	Expiry sweep → `{"expired": n}`	`403`

Meta

`GET /api/health` — no auth — `{"status":"ok","time":"..."}`

---

10. Important Code Concepts

Only concepts this project actually uses.

Python / backend

Type hints & `from __future__ import annotations` — Every function is annotated, and FastAPI *uses* the annotations at runtime to validate and document. The `__future__` import makes annotations lazy strings, which is what lets `Mapped[Donation]` reference a class defined further down the file.

Decorators — `@router.post(...)`, `@property`, `@lru_cache`, `@field_validator`, `@dataclass`. A decorator is a function that takes a function and returns a replacement. Be able to say that in one sentence.

Closures — `require_roles(*roles)` returns an inner function that has captured `roles` from the enclosing scope. This is the mechanism behind the whole authorization system. **A guaranteed interview question.**

Generators and `yield` — `get_db` yields a session and closes it in a `finally`. FastAPI treats the code after `yield` as teardown that runs once the response is done.

Dependency injection — `Depends(...)`. FastAPI builds the dependency graph, resolves it per request, and caches each dependency within a request. It is what keeps auth out of the body of every handler.

Context managers — `@asynccontextmanager` for the app lifespan; `with _session() as db` in the CLI.

Enums — `class UserRole(str, enum.Enum)`. Inheriting from `str` means the member is *also* a string, so it serialises to JSON directly and compares to `"donor"`.

The walrus operator `:=` — used in comprehensions in `metrics.py` and `matching.py` to bind and test in one expression:

```
[m for d in donations if (m := _minutes_between(...)) is not None]
```

`frozenset` — `SELF_SIGNUP_ROLES` is immutable, so the set of self-registerable roles cannot be mutated at runtime.

OOP and inheritance — `Base` → models; `Schema` → all Pydantic schemas; `RequirementOut(RequirementCreate)` extends rather than repeats; `UserAdminOut(UserOut)` adds admin-only fields; `NetworkError(ApiError)`.

`TypeDecorator` — the `UtcDateTime` custom column type (§6.5).

Properties — `reliability_score`, `initials`, `recipient_id` are computed, not stored, so they cannot go stale.

SQL / ORM

ORM vs raw SQL — objects instead of rows; parameterisation for free.

Sessions and the unit of work — a `Session` accumulates changes and flushes them as one transaction on `commit()`. Understand `flush()` (send SQL, get the generated id, stay in the transaction) versus `commit()` (make it permanent). `register()` uses exactly this: `flush` to learn `user.id`, then insert the profile row, then one `commit`.

Transactions — every mutating endpoint is a single transaction, so a status event and the status change land together or not at all.

Eager vs lazy loading — `selectinload` is the N+1 fix (§6.7).

Cascades — `cascade="all, delete-orphan"` on events and requirements.

Indexes — see §6.4.

REST / HTTP

Resources and verbs — `GET` read, `POST` create or perform an action, `PATCH` partial update, `DELETE` remove. Note `POST /api/donations/{id}/status` is an *action* endpoint, not pure REST — defensible because a lifecycle transition is a command, not a field assignment.

Status codes as used here — `200`, `201`, `401` (who are you), `403` (I know who you are, no), `404`, `409` (conflicts with current state), `422` (malformed input). **The 401/403 and 409/422 distinctions are classic interview questions and this codebase uses them correctly.**

Statelessness — no server-side session store; the token carries identity.

CORS — a browser rule, not a server one; the server merely declares which origins may read its responses.

TypeScript / React

Interfaces and structural typing — `ApiDonation` mirrors `DonationOut`.

Union types and literals — `DonationStatus` is a union of nine string literals, so a typo is a compile error.

Generics — `request<T>(path, options): Promise<T>` types the response at the call site; `run<T>(key, action)` in `useAction`.

Utility types — `Partial<Omit<ApiRecipient,'id'|'isVerified'>>`, `Exclude<UserRole,'admin'>` — the last one encodes "you cannot register as an admin" **in the type system**.

Promises and `async/await` — every API call. Know that `await` unwraps a promise and that a rejected promise becomes a thrown error inside `try/catch`.

Hooks — `useState`, `useEffect`, `useCallback`, `useMemo`, `useRef`, `useContext`, plus the custom `useAction`, `useMatchAnalysis`, `useIsMobile`.

`useRef` vs `useState` — a ref persists across renders **without** triggering one, and updates synchronously. The 401 re-entrancy guard and `useAction`'s `mounted` flag both need exactly that.

Effect cleanup and the `cancelled` flag — every data-fetching effect declares `let cancelled = false` and returns `() => { cancelled = true }`, so a response arriving after unmount or after the inputs changed is discarded. This prevents both the "setState on unmounted component" warning and a stale response overwriting a newer one.

Context — `createContext` + a provider + a `useX` hook that throws when used outside the provider (a nice fail-fast pattern worth pointing out).

Discriminating errors by class — `error instanceof ApiError` and the `isForbidden` / `isConflict` getters.

---

11. Critical Code Walkthroughs

Five pieces of code. If you understand these, you understand the project.

11.1 `update_status` — the lifecycle engine

`code/foodlink/routers/donations.py:165`

What it does: the single entry point for every state change a donation can undergo. Every rule about who may do what, when, lives here or in the tables it consults.

Step by step:

Load with relations — `_get_or_404` uses the `_loaded` query with `selectinload`, so the side effects and the response can read `recipient`, `volunteer.user`, and `events` without extra queries. `404` if absent.

Is the transition legal? `python if target not in ALLOWED_TRANSITIONS[donation.status]: → 409` The state machine is a dict lookup, not a chain of `if`s. Adding a state means adding a dict entry. `COMPLETED`, `CANCELLED`, and `EXPIRED` map to `set()` — **terminal, nothing can follow.**

Is the caller's role permitted? `python allowed_roles = TRANSITION_ROLES.get(target, set()) is_owning_donor = user.role is donor and donation.donor_id == user.id if user.role not in allowed_roles and not (target is CANCELLED and is_owning_donor): → 403` Note the shape: a role gate **plus** a narrow ownership exception. A donor is not in `TRANSITION_ROLES[CANCELLED]` in a general sense — they are permitted only for *their own* donation.

Side effects, per target. For `ACCEPTED`: - admins may name any `recipientId`; an NGO always resolves to its own - an NGO naming a *different* organisation → 403 - unresolved → 422; unverified → 403 - bind `recipient_id`, increment `accepted_donations` - **freeze `match_score`** at the value that justified this decision

For `VOLUNTEER_ASSIGNED`: `python if donation.volunteer_id not in (None, volunteer.id): → 409` The race guard. For `COMPLETED`: increment the recipient's `completed_donations` and the courier's `completed_deliveries` — which is what makes `reliability_score` mean something.

5. **Record and commit** — append the `StatusEvent` (server-stamped `occurred_at`), set the status, one `commit()`.

Why it is built this way: the transition table and the role table are **data**, so the rules are readable in one place and testable without HTTP. The alternative — nested conditionals — would scatter the same logic across the function and make "can a volunteer cancel?" a question you answer by reading code instead of a table.

Common bugs to be ready to discuss: - Adding a status to `DonationStatus` but forgetting `ALLOWED_TRANSITIONS` → `KeyError` and a 500 on the *previous* status's lookup. - Adding one but forgetting `TRANSITION_ROLES` → `.get(target, set())` returns empty, so **nobody** can perform it. Fails closed, which is the right direction, but silently. - Mutating `donation.status` before `_record` reads it as `from_status` would corrupt the history — the current ordering inside `_record` is load-bearing.

The concurrency caveat, stated honestly: the courier guard is a read-then-write with no row lock and no unique constraint. Two couriers hitting it simultaneously could both read `volunteer_id IS NULL` and both write. SQLite serialises writes so this is very unlikely today, but on Postgres it is a real TOCTOU race. The fix is `SELECT ... FOR UPDATE` or a conditional `UPDATE ... WHERE volunteer_id IS NULL` and checking the row count. **Raise this yourself — it is exactly the kind of thing "what happens if two users modify the same data?" is fishing for.**

How to modify it: to add a `REJECTED` state — add the enum member, add it to `ALLOWED_TRANSITIONS[ACCEPTED]`, give it an entry in `ALLOWED_TRANSITIONS` (probably `set()`), add `TRANSITION_ROLES[REJECTED] = {ngo, admin}`, add the literal to the TypeScript union, and handle it in `StatusBadge`. Six small, obvious places — which is the payoff of the table-driven design.

11.2 `score_pair` — the matching engine

`code/foodlink/matching.py:105`

What it does: scores one donation/recipient pair 0–100, or returns `None` if the pair is ineligible.

Two hard gates first (these return `None`, they do not score low):

```
if not recipient.is_verified:      return None   # trust
if recipient.latitude is None:     return None   # unmappable
if distance > radius_km:           return None   # out of range
```

Gating rather than scoring is deliberate: a suggestion the recipient could not legally act on would be a false promise to the donor.

Then five normalised criteria, each 0–100:

Criterion	Weight	Logic
`distance`	0.25	Linear decay to 0 at the 8 km radius
`quantity`	0.25	Peaks at exactly 100% of capacity; `ratio ≤ 1 → 40 + 60·ratio`; overflow penalised at **twice** the slope (`100 − 120·(ratio−1)`)
`capacity`	0.20	Headroom left after taking it; 0 if it would overflow
`deadline`	0.15	Slack after subtracting travel time at an assumed 20 km/h; 2 h of slack = 100
`reliability`	0.15	The recipient's completion record (85 prior when < 3 accepted)

```
overall = Σ (score_i × WEIGHTS[i])       # weights sum to exactly 1.0
```

Worked example — 120 meals, kitchen 1.2 km away, capacity 150, 6 h to the deadline, reliability 92:

```
distance   : 100 × (1 − 1.2/8)          = 85
quantity   : ratio 0.80 → 40 + 60×0.80  = 88
capacity   : 100 × (1 − 0.80×0.5)       = 60
deadline   : travel 3.6 min, slack ■2 h = 100
reliability:                              92
overall = 85(.25) + 88(.25) + 60(.20) + 100(.15) + 92(.15)
        = 21.25 + 22.00 + 12.00 + 15.00 + 13.80 = 84.05 → 84
```

Practise this by hand. "Walk me through how a score of 84 was produced" is the single most likely deep question on this project.

Why a weighted sum and not ML: there is no labelled training data (the platform has no history until it is used); the score must be *explainable* to a recipient; and a marker can verify it by hand. The file says the swap point is `score_pair` alone — the router and response shape would not change.

Weaknesses to concede: the weights are chosen by judgement, not tuned. The 20 km/h travel assumption is a constant, not a routing API. `haversine` is straight-line distance, so a river between two points is invisible. `COLD_STORAGE` is **defined but never used** — storage type does not actually gate anything, despite the comment saying it should. That last one is a genuine dead-code finding worth naming.

11.3 `get_current_user` — authentication

`code/foodlink/security.py:41`

```
try:
   payload = jwt.decode(token, settings.secret_key, algorithms=[ALGORITHM])
   user_id = int(payload["sub"])
except (jwt.PyJWTError, KeyError, ValueError):
   raise credentials_error from None

user = db.get(User, user_id)
if user is None or not user.is_active:
   raise credentials_error
return user
```

Four things to notice:

1. **`algorithms=[ALGORITHM]` is pinned.** Without an explicit list, a library can be tricked into honouring the `alg` header of the token itself — including `none`. Pinning it is what makes the signature check meaningful.
2. **One error object for every failure.** Bad signature, expired, malformed, missing `sub`, deleted user, suspended user — all produce the identical 401. No information leaks about *why*.
3. **`from None` suppresses exception chaining**, so a stack trace cannot expose the underlying JWT error.
4. **The database read is the design.** The token's `role` claim is never trusted; the live row wins. One indexed PK lookup per request buys immediate suspension and immediate role changes.

How to modify: to add a `token_version`, put it in the payload at mint time, add the column to `users`, and compare here — one extra condition gives you real revocation.

11.4 `request<T>` — the frontend API boundary

`frontend/src/lib/api.ts:107`

Four responsibilities in one function: attach the token, catch a true network failure as `NetworkError`, treat 401 as a global session end, and turn any other non-ok response into an `ApiError` carrying the server's own sentence.

The ordering is the interesting part:

```
fetch → catch          ⇒ NetworkError (never reached the server)
401 & !anonymous       ⇒ clear token, fire global handler, throw
204                    ⇒ undefined (no body to parse)
parse body (tolerant of empty/non-JSON)
!ok & ≥500 & no JSON   ⇒ NetworkError  (dead backend behind the proxy)
!ok                    ⇒ ApiError(status, extractDetail(...))
ok                     ⇒ T
```

`anonymous: true` on login is essential: a 401 there means "wrong password", not "your session expired", and must not trigger the global sign-out.

11.5 `conftest.py` — how the tests get an isolated database

`code/tests/conftest.py:15`

```
engine = create_engine("sqlite://",
                       connect_args={"check_same_thread": False},
                       poolclass=StaticPool)
```

Three deliberate choices:

- **`"sqlite://"` with no path** = a purely in-memory database. Fast, and nothing to clean off disk.
- **`StaticPool`** is the trick. An in-memory SQLite database exists *per connection*; the default pool would hand out a new connection and the tables would vanish. `StaticPool` reuses one connection, so the test and the request share a database.
- **`check_same_thread: False`** because `TestClient` runs the app in a threadpool.

Then `app.dependency_overrides[get_db] = override_get_db` swaps the real session for the test one **without the application code knowing** — the clearest payoff of dependency injection in the whole project, and a great thing to point at when asked "why does DI matter?".

`admin_token()` deserves a mention too: since the API deliberately has no path to a first administrator, the fixture inserts one directly — *exactly as the CLI does* — then authenticates normally through the API. The test respects the security boundary rather than punching through it.

---

12. End-to-End User Flows

12.1 Login

```
User types email + password → clicks "Sign in"
  ↓
Login.tsx: local useState; minimal client validation
  ↓
AuthContext.signIn(email.trim(), password)
  ↓
api.login() → POST /api/auth/login (FORM-encoded, anonymous:true)
  ↓
■■ Vite proxy in dev (/api → :8000), same-origin, no CORS ■■
  ↓
FastAPI login()  [auth.py:79]
  ↓ SELECT users WHERE email = lower(username)
  ↓ bcrypt.checkpw(plain, stored_hash)
  ↓ user.is_active?
  ↓ create_access_token → HS256 { sub, role, exp: +720min }
  ↓
200 { accessToken, user }
  ↓
setToken() → localStorage['foodlink.token']
setUser(toUser(response.user))
  ↓
Login.tsx navigates to HOME_PATH[user.role]  (or the remembered `from`)
  ↓
ProtectedRoute sees a user with a permitted role → <Outlet/>
  ↓
Layout mounts; AppContext loads donations/stats/etc.
  ↓
Dashboard renders
```

Failure branches: wrong password → `401` → `ApiError` → inline error, token untouched. Suspended → `403` with the deactivation sentence. Backend down → `NetworkError` → "Cannot reach the FoodLink server."

12.2 Posting a donation

```
Donor fills CreateDonation form (name, category, quantity, unit,
storage, location + coordinates, prepared time, pickup deadline)
  ↓
AppContext.createDonation(draft)
  ↓
POST /api/donations   Bearer <jwt>
  ↓
Depends(require_roles(donor, admin))    ← 403 if an NGO tries
Pydantic DonationCreate                 ← 422 on quantity ≤ 0, bad coords
  ↓
deadline in the past? → 422
  ↓
INSERT donations (status=AVAILABLE)
flush() → id
INSERT status_events (to=AVAILABLE, actor=donor)
  ↓
SELECT all recipients
rank_recipients(donation, recipients, radius=8km, limit=1)
  ■■ each recipient: verified? has coords? within radius?
  ■■ score the survivors, sort desc
  ↓
if any: donation.match_score = top.overall_score
       INSERT status_events (AVAILABLE→MATCHED, note="Top match: <name>")
       donation.status = MATCHED
  ↓
commit()  ← donation + both events atomically
  ↓
201 DonationOut
  ↓
AppContext refetches; success toast; donor sees the card with a match score
```

Remember: `MATCHED` assigns nobody. `recipientId` is still null.

12.3 Accepting (the NGO side)

12.4 Pickup and delivery (the volunteer side)

```
ACCEPTED
  ↓ volunteer claims
POST /status {"status":"VOLUNTEER_ASSIGNED"}
  ■■ role must be volunteer|admin
  ■■ SELECT volunteers WHERE user_id = me   (422 if none)
  ■■ donation.volunteer_id ∉ (None, me) → 409 "Another courier has already
  ■                                          claimed this pickup"
  ■■ donation.volunteer_id = me
  ↓ collects the food
POST /status {"status":"PICKED_UP"}          (volunteer|admin)
  ↓ delivers
POST /status {"status":"DELIVERED"}          (volunteer|admin)
  ↓ THE RECIPIENT confirms — not the courier
POST /status {"status":"COMPLETED"}          (ngo|admin)
  ■■ recipient.completed_donations += 1     → feeds reliability_score
  ■■ volunteer.completed_deliveries += 1
  ↓
Terminal. ALLOWED_TRANSITIONS[COMPLETED] = set()
```

12.5 Organisation verification

```
NGO registers → Recipient row created with is_verified = FALSE
  ↓
It CAN sign in, browse, post requirements
It CANNOT accept (403) and is EXCLUDED from every match ranking (score_pair → None)
  ↓
Admin opens AdminOrganizations → POST /api/admin/recipients/{id}/verify
  ↓
is_verified = TRUE
  ↓
Now rankable and now able to accept
```

The two consequences of `is_verified` are enforced in **two different files** — `matching.score_pair` (invisible in rankings) and `donations.update_status` (refused on accept). Both are needed: ranking without the accept check would let a client bypass it.

12.6 The metrics pipeline

```
Every transition, all through the app's life
  ↓
status_events accumulates (append-only, server-stamped)
  ↓
GET /api/metrics
  ↓
SELECT all donations + selectinload(events)      full scan into memory
  ↓
partition: completed / expired / active
  ↓
time-to-claim  = median(ACCEPTED.occurred_at − created_at)
handover       = median(DELIVERED.occurred_at − ACCEPTED.occurred_at)
rescue rate    = |completed where COMPLETED ts ≤ pickup_deadline| / |completed ∪ expired|
expiry loss    = |expired| / |completed ∪ expired|
  ↓
null instead of 0 when there is no history — an honest "unknown"
```

12.7 Session expiry mid-use

```
Token expires (or an admin suspends the account)
  ↓
The next request from any screen → 401
  ↓
api.ts: setToken(null); onUnauthorized()
  ↓
AuthContext: expiring ref guards against N parallel 401s announcing N times
  ↓
setUser(null) + expiredMessage = "Your session ended. Please sign in again."
  ↓
ProtectedRoute sees user === null → <Navigate to="/login" state={{from}} />
  ↓
Login screen explains why; signing in returns them where they were
```

---

13. Error Handling & Debugging

13.1 A systematic process

```
1. WHERE did it fail?   Browser console · Network tab · uvicorn terminal · DB
2. WHAT is the status?  401 · 403 · 404 · 409 · 422 · 500 · no response at all
3. READ the `detail`.   This backend writes real sentences. They are usually
                        the answer, not a generic "Bad Request".
4. REPRODUCE minimally. curl or /docs — take the frontend out of the loop.
5. NARROW the layer:    frontend state → network → route → auth → logic → DB
6. VERIFY the fix:      pytest code/tests   (37 tests, ~19 s)
```

The highest-leverage habit for this project: open `/docs` (Swagger) and hit the endpoint directly with the "Authorize" button. It removes the entire frontend from consideration in about ten seconds.

13.2 Symptom → cause tables

"Login doesn't work"

Observation	Likely cause	Where to look
`401 Incorrect email or password`	Wrong password, or no such account	`python -m foodlink.cli list-admins`; check `users`
`403` with a deactivation message	`is_active = false`	Admin panel, or set it directly in the DB
`422` on login	Sending JSON instead of form-encoded	Login **must** be `x-www-form-urlencoded`
`NetworkError` toast	Backend not running	Is uvicorn up on `:8000`?
Works in curl, not the app	Proxy misconfigured	`vite.config.ts` target vs the actual port
Login succeeds, next request 401	Secret key changed between mint and verify	`FOODLINK_SECRET_KEY` differs, or the server restarted with a different env

`401 Unauthorized`

Means **"I don't know who you are."** Causes, in order of likelihood: 1. No `Authorization` header — check the Network tab request headers 2. Token expired (12 h) — decode it on jwt.io and read `exp` 3. `FOODLINK_SECRET_KEY` changed since the token was issued → signature fails 4. Account suspended or deleted — `get_current_user` returns 401 for both 5. Malformed token in localStorage — clear it: `localStorage.removeItem('foodlink.token')`

`403 Forbidden`

Means **"I know who you are, and no."** This project has five distinct 403s and the `detail` tells you which:

Detail contains	Cause
`This action requires one of: ...`	`require_roles` — wrong role for the endpoint
`Your role cannot set a donation to X`	`TRANSITION_ROLES` — wrong role for that transition
`You can only accept ... your own organisation`	NGO named another organisation's id
`awaiting verification`	`is_verified = false` — an admin must verify
`This account has been deactivated`	Suspended, caught at login

`404 Not Found`

- Wrong id → confirm the row exists
- **Wrong URL** — remember the `/api` prefix and that donations are `/api/donations` (no trailing slash on the collection route)
- Router not mounted in `main.py`

`409 Conflict`

Unusually informative in this codebase: - `Cannot move a donation from X to Y` → check `ALLOWED_TRANSITIONS[X]` - `Another courier has already claimed this pickup` → `volunteer_id` is set - `An account with that email already exists` - `You cannot suspend or demote your own administrator account` - `This is the last active administrator`

`422 Unprocessable Entity`

Pydantic. The body lists exactly which field failed and why. Common ones here: `pickupDeadline` in the past · `role: "admin"` on register · password < 8 · `quantity <= 0` · coordinates out of range · **camelCase/snake_case mismatch** (the wire is camelCase — `foodName`, not `food_name`).

`500 Internal Server Error`

There is no custom handler, so the real traceback is in the **uvicorn terminal** — always look there first. Likely causes in this codebase: - A new `DonationStatus` member missing from `ALLOWED_TRANSITIONS` → `KeyError` - A schema/model mismatch after editing a model without recreating the DB - `None` where the code assumed a relation exists

Note the confusing interaction: `api.ts` converts a 500 with no JSON body into `NetworkError` ("Cannot reach the FoodLink server"). So a genuine backend *crash* can look like a backend that is *down*. If you see that toast and uvicorn is clearly running, **go read the uvicorn log** — it is a 500, not a connectivity problem.

"Database connection fails"

With SQLite there is no server to fail, so the realistic causes are: - **Wrong working directory.** `sqlite:///./foodlink.db` is *relative*. Running uvicorn from the repo root vs from `code/` gives you two different databases — which is exactly why this repo contains two `foodlink.db` files. **This is the most common "my data disappeared" cause here.** - File permissions, or a corrupted file (delete and re-seed) - With `DATABASE_URL` pointing at Postgres: wrong credentials/host, or the server is down. `pool_pre_ping=True` will surface a dead connection rather than hanging.

"Frontend isn't receiving data"

```
Network tab: did the request fire at all?
■■ No  → the component never called it; check the effect's dependency array
■■ Yes → what status?
     ■■ 200 but empty [] → the data genuinely isn't there; check `mine`
     ■                     and status filters, and seed the DB
     ■■ 200 with data but nothing renders → an adapter or a naming mismatch:
     ■     the wire is camelCase; check `lib/adapters.ts`
     ■■ 401/403 → auth, see above
     ■■ pending forever → backend hung, or the proxy target is wrong
```

"CORS error"

First: you should not see one in development at all. Vite proxies `/api` to `:8000`, so the browser only ever talks to one origin. A CORS error in dev means the frontend is calling the backend **directly** — i.e. `VITE_API_URL` is set when it should not be, or an absolute URL is hard-coded somewhere.

In production, CORS applies for real. The fix is `CORS_ORIGINS` on the backend:

```
CORS_ORIGINS="https://foodlink.example.com"
```

Remember it must be the **exact origin** — scheme, host, and port all matter; `http://` ≠ `https://`, and `localhost` ≠ `127.0.0.1`.

"Authentication suddenly breaks for everyone"

Almost always: **`FOODLINK_SECRET_KEY` changed.** Every previously issued token now fails its signature check. This happens when the server restarts without the env var, silently falling back to the built-in default (or away from it). It is the practical argument for the fail-fast fix in §8.4.

"Production works differently from development"

The genuine differences in this project:

	Development	Production
API origin	Vite proxy, same-origin	Real cross-origin
CORS	Never exercised	Actually enforced
Secret key	Insecure default	Must be set (nothing checks)
Database	SQLite file	Should be Postgres
Frontend	Vite dev server, HMR	Static build from `dist/`
Type errors	Warnings while running	**`npm run build` runs `tsc` and fails**
Routing	Vite handles deep links	**needs an SPA rewrite rule**

The classic production-only bug for this app: a hard refresh on `/donor/donations` returns 404 because the static host looks for that file. Any SPA needs a catch-all rewrite to `index.html`. It cannot happen in dev, so it is always discovered in production.

13.3 Practical commands

```
# Backend, from the repo root
.venv/Scripts/python.exe -m uvicorn foodlink.main:app --reload --app-dir code

# Tests
.venv/Scripts/python.exe -m pytest code/tests -q
.venv/Scripts/python.exe -m pytest code/tests -k lifecycle -vv   # one area
.venv/Scripts/python.exe -m pytest code/tests -x --pdb           # stop and inspect
```

```
# Frontend
cd frontend && npm run dev
```

```
# Inspect the database directly
sqlite3 code/foodlink.db ".tables"
sqlite3 code/foodlink.db "SELECT id,email,role,is_active FROM users;"
sqlite3 code/foodlink.db "SELECT id,status,pickup_deadline FROM donations;"
sqlite3 code/foodlink.db "SELECT * FROM status_events WHERE donation_id=42 ORDER BY occurred_at;"
```

```
# Bootstrap an administrator (run from code/)
python -m foodlink.cli create-admin --email you@example.com --name "You"
python -m foodlink.cli list-admins
python -m foodlink.cli reset-password --email you@example.com
```

Browser-side:

```
localStorage.getItem('foodlink.token')      // is there a token?
localStorage.removeItem('foodlink.token')   // force a clean sign-out
```

Paste the token into jwt.io to read `sub`, `role`, and `exp` — remember it is signed, not encrypted, so this always works.

---

14. Testing

14.1 What exists

Framework: pytest 8.3 with FastAPI's `TestClient` (httpx under the hood). **Location:** `code/tests/` — `conftest.py`, `test_api.py`, `test_auth_admin.py`. **Count: 37 tests. Verified passing:** `37 passed in 19.15s`. **Configuration:** `pyproject.toml` sets `pythonpath = ["code"]` so `import foodlink` resolves without installing the package.

The ~19-second runtime is almost entirely **bcrypt** — every `register()` helper hashes a real password at cost factor 12. That is the price of testing the real hashing path rather than a stub, and it is the right trade.

14.2 Test style — integration, not unit

Every one of the 37 tests goes through the **full HTTP stack**: routing, Pydantic validation, dependency injection, authorization, ORM, and a real (in-memory) database. There are **no mocks anywhere**.

Classifying them honestly: - **Integration tests: 37** (API-level, real DB, real bcrypt, real JWT) - **Unit tests: 0** — notably, `matching.py` is pure and trivially unit-testable but is only tested *through* the API - **E2E tests: 0** — no browser automation - **Frontend tests: 0** — none of any kind

14.3 What each area verifies

`test_api.py` (15 tests) — the core domain

Test	What it proves
`test_health`	The app boots and routes
`test_register_then_login`	The credential round trip works
`test_duplicate_email_rejected`	409 on a taken email
`test_wrong_password_rejected`	bcrypt actually rejects

`test_endpoints_require_a_token`	Anonymous access is refused
`test_creating_a_donation_stamps_it_and_matches`	Creation stamps an event **and** auto-ranks
`test_deadline_in_the_past_is_rejected`	The business rule, not just the schema
`test_matches_are_ranked_and_exclude_out_of_radius`	The radius gate works
`test_a_nearer_kitchen_outranks_a_further_one`	**The ranking is genuinely ordered** — the single most valuable matching test
`test_full_lifecycle_to_completion`	All seven states end to end
`test_illegal_transition_is_rejected`	`ALLOWED_TRANSITIONS` is enforced (409)
`test_a_donor_cannot_accept_on_a_kitchens_behalf`	Role gating on transitions
`test_second_courier_cannot_steal_a_claimed_pickup`	**The race guard** (409)
`test_metrics_report_time_to_claim`	Metrics derive from real events
`test_requirements_round_trip`	Requirements create and list

`test_auth_admin.py` (22 tests) — auth, admin, and trust

Grouped by what they defend:

Privilege escalation - `test_registration_cannot_mint_an_administrator` - `test_admin_endpoints_are_closed_to_every_other_role` - `test_admin_endpoints_reject_anonymous_callers` - `test_an_administrator_can_appoint_another`

Lockout prevention - `test_the_platform_cannot_be_left_without_an_administrator` - `test_an_administrator_can_be_suspended_once_another_exists`

Suspension semantics - `test_a_suspended_account_is_turned_away_at_login` - `test_a_suspended_accounts_existing_token_stops_working` ← **proves the per-request DB read matters**

The verification trust model - `test_ngo_registration_creates_an_unverified_organisation` - `test_an_unverified_organisation_cannot_accept` - `test_verification_unlocks_acceptance` - `test_an_organisation_cannot_verify_itself` - `test_unverified_organisations_are_not_ranked` ← both halves of the rule - `test_an_ngo_cannot_accept_for_another_organisation` - `test_an_admin_may_accept_on_an_organisations_behalf`

Self-service boundaries - `test_an_organisation_can_complete_its_own_profile` - `test_password_change_requires_the_current_one`

Maintenance - `test_the_expiry_sweep_closes_an_overdue_donation` - `test_the_expiry_sweep_is_administrator_only`

The CLI - `test_the_cli_creates_the_first_administrator` - `test_the_cli_promotes_an_existing_account` - `test_the_cli_refuses_to_duplicate_an_existing_account`

What is impressive about this suite, and worth saying out loud: the test names are *sentences describing security properties*, not `test_update_user_2`. `test_a_suspended_accounts_existing_token_stops_working` tells you what the system guarantees. That is documentation that cannot rot.

14.4 Missing coverage — be honest about this

Entirely untested

- **The whole frontend.** 83 TypeScript files, zero tests. No Vitest, no Testing Library, no Playwright. `tsc` in the build is the only safety net.
- **`matching.py` as a unit.** The scoring *functions* are never called directly, so the exact curve of `_quantity_score` at the overflow boundary, `_deadline_score` when slack is negative, and the `reliability_score` cliff at 3 accepted donations are all unverified in isolation.
- **The `UtcDateTime` decorator.** Given it exists to prevent a subtle timezone bug, having no test that a naive datetime survives a round trip as UTC is a real gap.
- **Concurrency.** The courier race is tested *sequentially* (second request after the first completed), which does not exercise the actual TOCTOU window.

Thin coverage

- Pagination (`limit`, the 500 cap)
- `mine=true` scoping for all three roles
- Requirement listing filters (`is_active`)
- Volunteer profile endpoints
- Every 422 branch
- `revoke_verification` and its effect on an already-accepted donation

Edge cases worth adding (good answers to "what would you test next?")

1. A donation whose deadline passes **while** it is `ACCEPTED` — the sweep only touches `AVAILABLE`/`MATCHED`, so it stays stuck. **Is that intended?**
2. `reliability_score` exactly at the 3-donation boundary (2 vs 3 accepted)
3. A recipient with `capacity` smaller than the donation → `_capacity_score` 0
4. Antimeridian / equator coordinates through haversine
5. A 72+ character password (bcrypt truncation)
6. Two `PATCH /api/admin/users` calls demoting the last two admins concurrently
7. `revoke_verification` on an organisation mid-lifecycle

14.5 No CI

Tests never run automatically. The only workflow, `.github/workflows/mkdocs.yml`, builds documentation. A commit that breaks all 37 tests merges to `master` with no signal.

The fix is genuinely small and is the highest-value process improvement available:

```
name: tests
on: [push, pull_request]
jobs:
 backend:
   runs-on: ubuntu-latest
   steps:
     - uses: actions/checkout@v4
     - uses: actions/setup-python@v5
       with: { python-version: '3.11' }
     - run: pip install -r code/requirements-dev.txt
     - run: pytest code/tests -q
 frontend:
   runs-on: ubuntu-latest
   steps:
     - uses: actions/checkout@v4
     - uses: actions/setup-node@v4
       with: { node-version: '20', cache: 'npm',
               cache-dependency-path: frontend/package-lock.json }
     - run: npm ci --prefix frontend
     - run: npm run build --prefix frontend   # tsc + vite build
```

---

15. Deployment & Production

15.1 The honest status

This project has no deployment configuration of any kind. Verified absent: Dockerfile, docker-compose, Procfile, nginx config, Kubernetes manifests, Vercel/Netlify/Railway/Render config, systemd units, and any deployment CI. The only workflow deploys **documentation**.

Say this plainly in an interview. Then describe what you *would* do — that answer is worth more than a config file you cannot explain.

15.2 How it runs today (development)

```
# Terminal 1 — backend
.venv/Scripts/python.exe -m uvicorn foodlink.main:app --reload --app-dir code
# → http://127.0.0.1:8000  (docs at /docs)

# Terminal 2 — frontend
cd frontend && npm install && npm run dev
# → http://localhost:5173, proxying /api to :8000
```

```
# First administrator (from code/)
python -m foodlink.cli create-admin --email you@example.com --name "You"

# Optional demo data
python -m foodlink.seed
```

15.3 Build process

Frontend: `npm run build` = `tsc && vite build` → static assets in `frontend/dist/`. **A type error fails the build**, which is the only automated frontend gate that exists.

Backend: no build step. `pyproject.toml` describes the *template*, not this application, so the package is not actually installable as `foodlink` — it runs from source via `--app-dir code`.

15.4 Environment variables — the complete list

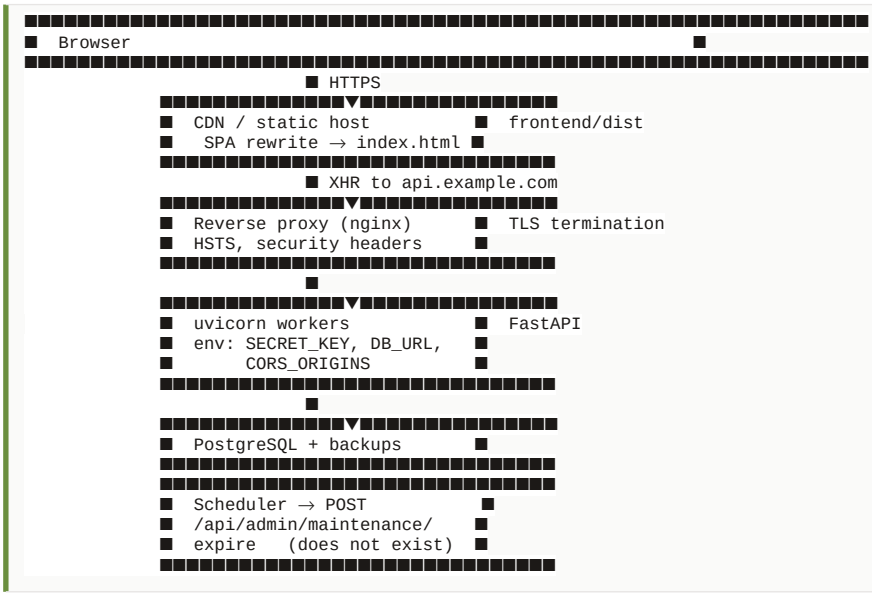
Every one, verified from `config.py`:

Variable	Default	Production requirement
`DATABASE_URL`	`sqlite:///./foodlink.db`	**Set to Postgres.** SQLite has a single writer and dies with the container's filesystem.
`FOODLINK_SECRET_KEY`	`dev-only-insecure-key-replace-me-in-deployment`	**MUST be set** to 32+ random bytes. `python -c "import secrets;print(secrets.token_urlsafe(48))"`
`ACCESS_TOKEN_MINUTES`	`720` (12 h)	Consider 60 or less
`CORS_ORIGINS`	`http://localhost:5173,http://127.0.0.1:5173`	**Set to the real frontend origin**, comma-separated
`MAX_MATCH_RADIUS_KM`	`8`	Tune to the city
`FOODLINK_ADMIN_PASSWORD`	unset	Only for scripted CLI bootstrap

Frontend build-time: `VITE_API_URL` (the API origin; empty means same-origin) and `VITE_API_PROXY` (dev proxy target). Vite inlines `VITE_*` at **build time**, so they are baked into the bundle — never put a secret in one.

The single most dangerous gap: `get_settings()` is `lru_cache`d and reads the environment once at import. If `FOODLINK_SECRET_KEY` is missing, the app **starts normally** with the published default. Nothing warns.

15.5 What deployment would require



A concrete checklist:

1. **Secrets** — generate and set `FOODLINK_SECRET_KEY`; never commit it.

2. **Database** — provision Postgres, set `DATABASE_URL`, `pip install psycopg2-binary`. **Add Alembic first** — shipping `create_all` to production means the first schema change loses data.
3. **CORS** — set `CORS_ORIGINS` to the exact frontend origin.
4. **Backend** — `uvicorn foodlink.main:app --host 0.0.0.0 --port 8000 --workers 4` (**multiple workers require Postgres**; SQLite would corrupt under concurrent writers).
5. **Frontend** — `npm run build`, serve `dist/` statically, **add the SPA rewrite**.
6. **TLS** — terminate at the proxy; add HSTS.
7. **Bootstrap** — run `create-admin` once against the production database.
8. **Scheduler** — a cron job with an admin token calling the expiry endpoint.
9. **Logs** — uvicorn writes to stdout; ship it somewhere. There is **no structured logging and no `logging` configuration in the app at all.**
10. **Monitoring** — `/api/health` exists and is a fine liveness probe, but it does **not** check the database, so it will report healthy while every request 500s. A readiness probe should run `SELECT 1`.

15.6 A Dockerfile if you needed one

Not present in the repo — this is a *recommendation*, and you should label it as such:

```
FROM python:3.11-slim
WORKDIR /app
COPY code/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY code/ ./code/
ENV PYTHONPATH=/app/code
EXPOSE 8000
CMD ["uvicorn","foodlink.main:app","--host","0.0.0.0","--port","8000"]
```

15.7 Production vs development — the real differences

Concern	Dev	Prod	Risk if ignored
CORS	Bypassed by proxy	Enforced	Every request blocked
Secret key	Default	Must be set	Forgeable admin tokens
Database	SQLite file	Postgres	Corruption, lost data
Workers	1, `--reload`	Several	SQLite corruption
Deep links	Vite handles	Needs rewrite	404 on refresh
Schema	Delete and recreate	Cannot	Data loss on change
Errors	Traceback in terminal	Nowhere	Blind debugging
Expiry sweep	Manual	Needs cron	Metrics drift

---

16. Performance & Scalability

16.1 What is already done well

N+1 is deliberately avoided. The `_loaded` query eager-loads four relationships with `selectinload`, turning 401 queries into 5 for a hundred-row list (§6.7).

Indexes match the access patterns — status, deadline, owner columns, and `status_events.donation_id` are all indexed (§6.4).

The match radius bounds the work. `MAX_MATCH_RADIUS_KM` exists explicitly so ranking does not grow with the total number of organisations… **in intent**. See the bottleneck below for why it does not yet deliver that.

A limit cap exists. `limit` defaults to 100 and is capped at 500; `/matches` is capped at 25.

16.2 Bottlenecks, in order of severity

1. `GET /api/metrics` loads the entire donations table into memory

```
donations = list(db.scalars(select(Donation).options(selectinload(Donation.events))))
```

Every donation and every status event, then medians computed in Python. At 1,000 donations this is fine. At 100,000 donations with ~6 events each, it is 600,000 rows into memory on every dashboard load. **Fix:** compute the aggregates in SQL (`COUNT`, `AVG`/percentile, `GROUP BY`), or maintain a rollup table refreshed periodically. Medians are the awkward part — Postgres has `percentile_cont`.

2. Ranking loads all recipients, ignoring the radius in SQL

```
recipients = list(db.scalars(select(Recipient)))     # ← no WHERE at all
ranked = rank_recipients(donation, recipients, radius_km=8)
```

The radius filter happens **in Python, after every row is already loaded**. So the cost is O(all organisations) per donation posted, and per `/matches` call. The comment in `config.py` says the radius "bounds the ranking work as the number of organisations grows" — **it does not, as currently written.** Naming this gap between stated intent and implementation is a strong interview move. **Fix:** a bounding-box `WHERE` clause on latitude/longitude (cheap, indexed, no extensions needed), then haversine on the survivors. At real scale, PostGIS with a GiST index.

3. SQLite has one writer

The whole database is locked during a write. Concurrent writes serialise, and a busy period produces "database is locked" errors. This is the hard ceiling on concurrency and the reason multiple uvicorn workers are unsafe today.

4. No pagination, only a cap

`limit` with no `offset` and no cursor. You can ask for the first 500 donations; there is no way to ask for the next 500. The UI therefore cannot page.

5. `Donation.timestamp_of` is a linear scan per call

```
for event in self.events:
   if event.to_status == status: return event.occurred_at
```

`metrics` calls this up to four times per donation, each scanning that donation's events. Small constants, but it is O(donations × events) overall.

6. Base64 images in a text column

`image_url` accepts data URLs with no size cap, so a single row can be megabytes — and every list response carries them all. **Fix:** object storage, store a URL.

7. Write-then-refetch doubles round trips

Every mutation is a write plus a re-read. Correct (§4.2), but chatty.

8. No caching anywhere

No Redis, no HTTP cache headers, no memoisation beyond `lru_cache` on settings. Every request recomputes everything.

9. No frontend code splitting

`App.tsx` imports every page and both the desktop and mobile trees eagerly, so the initial bundle carries all four portals. `React.lazy` per portal would be a clear win.

16.3 Scaling: 10 → 1,000 → 100,000 users

10 users — today's design is correct

SQLite, one uvicorn process, no cache. Everything is instant. Metrics scan a few hundred rows. **Nothing needs to change, and being able to say "this is appropriately built for its stage" is a mature answer.**

1,000 users (~50k donations)

What breaks first: 1. **SQLite write contention** → "database is locked" during busy periods 2. **Metrics** now scan ~300k event rows per dashboard load 3. **No pagination** — lists are truncated at 500 with no way forward 4. **The expiry sweep** still requires someone to remember to call it

Required changes: - Move to **Postgres** (`DATABASE_URL` only — the code already supports it) - **Alembic** before anything else changes - Multiple uvicorn workers behind nginx - Cursor pagination - Cache `/api/metrics` for 60 seconds - A real scheduler for expiry - Rate limiting

100,000 users (millions of donations)

Now the architecture itself has to change:

Problem	Change
Metrics scan	Pre-aggregated rollup tables or a warehouse; never scan live
Geographic ranking	**PostGIS** + GiST index; bounding box then exact distance
Read load	Read replicas; route all `GET`s to them
`status_events` growth	Partition by month; archive cold partitions
Ranking latency	Move matching to a background worker; notify results
Single backend	Horizontal scaling — the app is **already stateless**, which is why this works: JWT auth means no sticky sessions
Polling for updates	WebSockets or SSE for live donation feeds
Hot reads	Redis for recipient lists and metrics
Coupled lifecycle	Emit domain events; decouple notifications from transitions

The architectural property worth highlighting: because authentication is a stateless JWT and there is no server-side session store, the API scales horizontally by adding processes. That was a real design win, not an accident.

What would NOT need to change: the domain model. `status_events` as an append-only ledger is exactly what you want at scale — it is the standard shape for event-sourced analytics. The table-driven state machine also scales fine as pure logic.

---

17. Architecture Decisions & Tradeoffs

17.1 SQLite by default, Postgres-capable

Chosen: `sqlite:///./foodlink.db`, swappable via `DATABASE_URL`. **Why:** a fresh clone runs with zero setup — no server, no credentials, no container. For a graded coursework project where a marker must run it in five minutes, that is decisive. **Alternatives:** Postgres from day one (realistic, but every developer and marker needs it running); MongoDB (a poor fit — this data is deeply relational and the metrics are joins). **Advantages:** zero setup, trivially resettable, tests run in-memory. **Disadvantages:** one writer; no real concurrency; no timezone type (hence the `UtcDateTime` workaround); FKs unenforced by default. **When Postgres wins:** the moment there is a second concurrent writer, or any data you cannot afford to lose. **The migration is one environment variable** because SQLAlchemy abstracts the dialect — that forethought is the point.

17.2 An explainable weighted sum, not machine learning

Chosen: five weighted, normalised criteria with published weights and per-criterion reasons. **Why:** there is no training data before the platform is used; a recipient must be able to see *why* they were ranked; and it can be verified by hand. **Alternatives:** learning-to-rank (needs labelled outcomes that do not exist); a simple nearest-neighbour rule (ignores capacity, deadline, reliability); a constraint solver (heavier, and this is not a global-optimum problem). **Advantages:** explainable, debuggable, testable, tunable by argument. **Disadvantages:** weights are judgement, not evidence; no learning from outcomes; assumes a linear tradeoff between criteria. **When ML wins:** once there is real acceptance/completion history, a learned ranker could discover that reliability matters far more than distance. The code is structured for exactly that swap: **replace `score_pair`; the router and the response shape do not change. Say this in interviews:** "It is a heuristic, not a model, and that was deliberate."

17.3 An append-only event table instead of timestamp columns

Chosen: `status_events`. **Why:** metrics must be derived from evidence; the audit trail needs an actor; new states must not require migrations; re-entering a state must be representable. **Alternatives:** timestamp columns per state (simpler reads, but a migration per state, no actor, no repeats); a full event-sourcing framework (far too heavy). **Advantages:** complete history, honest metrics, extensible. **Disadvantages:** reading one timestamp is a scan of the donation's events; the table grows fastest of any in the schema. **When columns win:** if you only ever needed one timestamp and never an audit trail. **This is the strongest design decision in the project — lead with it.**

17.4 A table-driven state machine

Chosen: `ALLOWED_TRANSITIONS` + `TRANSITION_ROLES` as module-level dicts. **Why:** the rules are readable in one screen, testable without HTTP, and adding a state touches a known small set of places. **Alternatives:** nested conditionals in the handler (scatters the logic); a state-machine library (a dependency for something a dict does); database CHECK constraints (enforced everywhere, but unreadable and hard to change). **Disadvantages:** enforced only in the application — anything writing directly to the database can violate it. A missing dict entry fails at runtime, not at import.

17.5 React Context instead of Redux / React Query

Chosen: two Contexts, hand-rolled fetch, write-then-refetch. **Why:** the app has one screenful of shared state and a modest number of mutations. Redux would add actions, reducers, and a store for state that fits in a `useState`. **Alternatives:** Redux Toolkit (excellent devtools and time-travel, real boilerplate); **React Query** (genuinely the strongest alternative here — caching, background refetch, and request deduplication would directly fix the chattiness of write-then-refetch); Zustand (lighter than Redux, still a dependency). **Disadvantages:** every consumer of a Context re-renders when any part of it changes — so a toast can re-render donation lists. No caching, no dedup, no stale-while-revalidate. **When to switch:** if the app grew more screens sharing more state, **React Query is the change I would make first.** Have that answer ready.

17.6 JWT in localStorage instead of an httpOnly cookie

Chosen: bearer token in `localStorage`. **Why:** the session survives a reload; there is no CSRF surface; the API stays stateless and therefore horizontally scalable. **Alternatives:** an `httpOnly` cookie (immune to XSS *exfiltration*, but introduces CSRF and needs `SameSite` plus CSRF tokens); in-memory only (safest, but signs the user out on every refresh). **Advantages:** simple, stateless, no CSRF. **Disadvantages:** XSS-readable; **no revocation** — logout is client-side only. **The mitigation that already exists:** re-reading `is_active` per request means an admin can end a session immediately. That is a partial answer to revocation and worth naming.

17.7 No service layer

Chosen: business logic inside the router functions. **Why:** at this size, a `DonationService` that only forwards to the ORM would be indirection without insulation. Pure logic that *was* worth extracting (`matching`, `serialize`, `security`) **was** extracted. **Disadvantages:** `update_status` is ~80 lines mixing HTTP concerns with domain rules; the logic cannot be reused outside HTTP; testing it requires a request. **When to split:** as soon as the same transition must be driven from somewhere other than an HTTP request — a scheduled job, a queue consumer, the CLI. **The honest answer:** "There isn't one, deliberately, and here is the line I'd cut along if it grew."

17.8 `create_all` instead of migrations

Chosen: `Base.metadata.create_all` at startup. **Why:** zero setup for a marker or a fresh clone. **Disadvantages: does not alter existing tables.** No history, no rollback. Not production-viable. **The code concedes it:** *"Introduce Alembic before the schema has to change without dropping data."* **When to change: before the first production deployment.** Do not defend this one — concede it immediately and describe the fix.

17.9 camelCase on the wire via `alias_generator`

Chosen: Pydantic's `to_camel` on a shared `Schema` base. **Why:** Python is idiomatically `snake_case`, TypeScript `camelCase`. The generator translates in one place so neither side compromises and no component does manual mapping. **Alternatives:** snake_case everywhere (un-idiomatic JS); manual mapping in `adapters.ts` (repetitive and error-prone). **Disadvantage:** the same field has two names, so grepping the full stack for `food_name` misses `foodName`. Worth knowing when debugging.

17.10 A separate mobile component tree

Chosen: 26 dedicated mobile screens at a separate `/m/*` URL space. **Why:** phone flows genuinely differ — bottom navigation, a camera-first capture — and CSS breakpoints alone produce a squeezed desktop layout. **Alternatives:** responsive CSS only (one tree, less control); React Native (a whole second application). **Disadvantages: two sets of screens to keep in sync** — a real maintenance

cost and the most likely source of "it works on desktop but not mobile" bugs. And because entry is by URL rather than by viewport, **nothing sends a phone visitor to `/m/` automatically** — the `useIsMobile` hook that would have done it exists but is never imported. Either wire it up at the router, or treat `/m/` as a deliberate, separately-linked experience. Right now it is neither, which is the honest state to describe.

---

18. What You MUST Personally Know

Tier 1 — MUST KNOW (you will be asked, and a wrong answer is fatal)

#	Topic	Why
1	**The nine-state lifecycle and `ALLOWED_TRANSITIONS`**	The core domain model. Draw it from memory.
2	**`update_status` end to end**	The most complex function; every authorization idea meets here.
3	**Authentication: bcrypt → JWT → `get_current_user`**	The most-asked area in any interview.
4	**Why `get_current_user` re-reads the user from the DB**	The best auth decision in the project. Immediate suspension.
5	**The four authorization layers** (role, ownership, lifecycle, trust)	Shows you think in layers, not `if role == admin`.
6	**`status_events` — why an append-only table, not timestamp columns**	The strongest design decision. Lead with it.
7	**`score_pair`: five criteria, weights, gates**	Be able to compute a score by hand.
8	**That the matching is a heuristic, NOT machine learning**	Claiming ML would be caught immediately and would end the interview badly.
9	**`is_verified` and its two enforcement points**	Ranking *and* accept. Explains why both are needed.
10	**Why `admin` cannot self-register, and the CLI bootstrap**	A genuinely thoughtful piece of design.
11	**401 vs 403 vs 409 vs 422 — as this API uses them**	Basic HTTP fluency, and this project models it correctly.
12	**The N+1 problem and `selectinload`**	Near-universal question, and you have a concrete answer.
13	**The weaknesses: default secret key, broad read access, no rate limiting, no migrations, no CI**	Naming these yourself is a strength; being caught by them is a failure.

Tier 2 — SHOULD KNOW (expect these on follow-ups)

#	Topic	Why
14	The `UtcDateTime` decorator and the timezone bug it prevents	Shows real depth about a non-obvious failure.
15	`require_roles` as a closure / dependency factory	The mechanism behind all authorization.
16	`get_db` as a generator dependency	Explains guaranteed cleanup and DI.

17	The `StaticPool` in-memory test database + `dependency_overrides`	The clearest payoff of DI.
18	Write-then-refetch vs optimistic updates	A tradeoff you chose; defend it.
19	Why `ProtectedRoute` is UX, not security	A trap question. Never call it a security control.
20	The Vite proxy and why dev has no CORS	Explains a whole class of "works in dev" issues.
21	`useAction`'s keyed pending state	Concrete UX reasoning.
22	The global 401 handler and its `useRef` guard	Shows you understand refs vs state.
23	`reliability_score` and the 85 cold-start prior	A judgement call you must be able to justify.
24	The last-admin / self-demotion guards	Thinking about operational failure modes.
25	`flush()` vs `commit()` in `register`	Transaction fluency.
26	What the 37 tests cover, and that there are zero frontend tests	Honesty about coverage.
27	Every environment variable and its default	Deployment competence.
28	Why `MATCHED` assigns nobody	A subtlety interviewers probe.

Tier 3 — GOOD TO KNOW (conceptual is enough)

#	Topic
29	`alias_generator=to_camel` and the two-name problem
30	`extractDetail`'s handling of Pydantic's error list
31	The mobile component tree and `useIsMobile`
32	The Tailwind palette retune (warm stone/moss/clay)
33	`serialize.donation_out` computing `distanceKm` live
34	The haversine formula (concept, not derivation)
35	`lru_cache` on `get_settings`
36	`cascade="all, delete-orphan"`
37	The stale C++ Makefile and template `pyproject.toml`
38	The seed script's relative deadlines
39	That `COLD_STORAGE` and `useIsMobile` are defined but never used (dead code)
40	That `Volunteer.rating` is never written by any API path
41	That the mobile UI is a separate `/m/*` URL space, not a viewport branch

19. Interview Questions

56 project-specific questions. Every answer is grounded in the actual code.

Architecture (Q1–Q7)

Q1. Walk me through the architecture of your project. A. A React SPA talks over HTTP/JSON to a FastAPI backend, which uses SQLAlchemy against SQLite (Postgres-capable through one environment variable). The frontend has two Contexts — `AuthProvider` for identity and `AppProvider` for domain data — deliberately split because identity must settle before we know what we're allowed to fetch. All HTTP goes through one module, `lib/api.ts`, which attaches the bearer token and normalises errors. On the backend, five routers sit behind FastAPI's dependency injection: `get_db` for the session, `get_current_user` for authentication, `require_roles` for authorization. Domain logic that was worth isolating — matching, serialisation, security — lives in its own modules. There are **no external services at all**: no email, no SMS, no third-party APIs. **Follow-up:** *Why no service layer?* → §17.7 — at this size it would be indirection without insulation; the pure logic that deserved extraction was extracted. I'd cut a service layer out the moment a transition needed driving from something other than an HTTP request.

Q2. Why FastAPI over Django or Flask? A. Three reasons that this codebase actually uses. Its dependency injection is what makes `require_roles(UserRole.ngo)` composable and lets me gate an entire router in one line. It derives validation from type hints, so `RegisterRequest` *is* the validation. And it generates OpenAPI docs for free, which is how I test endpoints without the frontend. Django would have brought an admin panel and an ORM I didn't need alongside SQLAlchemy; Flask would have meant assembling validation and DI myself. **Follow-up:** *What does Django give you that you now miss?* → A migration framework out of the box. I have no migrations, which is my biggest infrastructure gap.

Q3. Why is `admin` in every `ProtectedRoute` allow-list? A. An administrator can act on any donation through the API regardless, so blocking them at the router would only stop them *seeing* what they can already *do*. Being able to view a donor's or a kitchen's portal is most of what platform support consists of. It's documented in `App.tsx`. **Follow-up:** *Doesn't that violate least privilege?* → It doesn't widen privilege at all — the server-side permissions are identical either way. It widens *visibility* for a role that already had full access. If admins were split into tiers, I'd revisit it.

Q4. How do frontend and backend stay in sync? A. Three layers. Pydantic schemas define the wire shape with `alias_generator=to_camel`. TypeScript interfaces in `lib/api.ts` mirror those schemas exactly. `lib/adapters.ts` translates wire types into the app's own domain types, so a backend change has one place to land instead of leaking into forty components. It's manual, not generated — a real weakness. **Follow-up:** *How would you automate it?* → Generate the TypeScript client from the OpenAPI schema FastAPI already publishes. Then a backend rename becomes a compile error rather than a runtime `undefined`.

Q5. Why is the mobile UI a separate component tree? A. The phone flows genuinely differ — bottom navigation, a camera-first capture screen — not just narrower columns. It's mounted at its own URL space, `/m/*`, with a nested router and an inner per-portal role guard. **I should be precise: entry is by URL, not by viewport.** There's a `useIsMobile` hook in the folder, but it's never imported — dead code — so nothing redirects a phone visitor automatically. **Follow-up:** *So how does anyone reach it?* → Only by navigating to `/m/` directly, which is a genuine gap. I'd either wire `useIsMobile` into a redirect at the router, or treat `/m/` as a deliberately-linked experience. Right now it's neither. **Follow-up:** *Would you do it again?* → For the capture flow yes. For the list screens, responsive CSS would have been enough and I over-built.

Q6. What happens if the frontend and backend disagree about state? A. They can't for long, by design. Every mutation is write-then-refetch — the client posts, then re-reads the affected slice from the server. The server owns the lifecycle rules and the server-stamped history, so its answer is authoritative. I pay a round trip to make disagreement impossible. **Follow-up:** *When is that the wrong choice?* → When failures are rare, latency is user-visible, and a wrong guess is cheap — a "like" button. Not a food-custody transition.

Q7. Where is the business logic? A. Split by nature. Pure, reusable logic is in modules: `matching.py` (no DB access at all, so it's unit-testable), `serialize.py`, `security.py`. The lifecycle *rules* are data in `models.py` — `ALLOWED_TRANSITIONS` — and `donations.py` — `TRANSITION_ROLES`. The orchestration lives in the router functions. **Follow-up:** *Why are the transition tables in two different files?* → Fair challenge. `ALLOWED_TRANSITIONS` is about the domain, so it sits with the model; `TRANSITION_ROLES` is about the API's permissions. In hindsight I'd keep them adjacent, since you almost always edit both together.

Frontend (Q8–Q16)

Q8. Why Context instead of Redux? A. The shared state is one screenful — donations, recipients, volunteers, requirements, stats, toasts — with a modest number of mutations. Redux would have added actions, reducers, and a store for something two Contexts hold comfortably. I split them so identity resolves before data is fetched. **Follow-up:** *What breaks first as it grows?* → Re-render breadth. Every consumer re-renders when any part of a Context changes, so a toast can re-render donation lists. I'd move to React Query before Redux, because my real problem is server-state caching, not client-state complexity.

Q9. Explain `ProtectedRoute`. A. Four branches: show a splash while a stored token is being exchanged (otherwise the login screen flashes in front of someone who *is* signed in); redirect to `/login` with the attempted path remembered in route state if there's no user; redirect to their own portal if they're signed in with the wrong role; otherwise render `<Outlet/>`. **It is a UX affordance, not a security control** — it runs in the browser, where the user controls everything. The server checks the role on every request regardless. **Follow-up:** *So what would happen if I deleted it?* → The app would be ugly and full of 403s. Nobody would gain access to any data.

Q10. How does the app know the session expired? A. `lib/api.ts` holds a module-level `onUnauthorized` callback that `AuthContext` registers once. Any 401 from anywhere clears the token and fires it, so the whole app learns at once rather than each screen discovering it separately. There's a `useRef` re-entrancy guard because a dashboard fires several parallel requests and all of them come back 401 — without it the user gets five "session expired" messages. **Follow-up:** *Why `useRef` and not `useState`?* → The flag has to be readable and writable synchronously within one tick. A state update wouldn't be visible to the second 401 arriving in the same event-loop turn.

Q11. Why re-fetch the user on boot instead of caching them? A. Only the token is persisted. If the user object were restored from localStorage, a suspended or re-roled account could keep acting on a stale copy of its own permissions until it happened to refresh. `GET /api/auth/me` costs one request and guarantees the client's view of its own role is current. **Follow-up:** *Doesn't that slow the first paint?* → Yes, by one request, which is why `isLoading` is initialised from whether a token exists at all — anonymous visitors never see the splash.

Q12. Explain `useAction`. A. Every action button has the same three states — idle, in flight, failed — and the same obligation to report the server's own message. `useAction` does that once. The important detail is that it tracks a **key**, not a boolean: on a list of twenty rows sharing one handler, a boolean would spin all twenty, so only the clicked row's key spins while `isBusy` disables the rest. It also tracks `mounted` in a ref so an unmounted component doesn't set state. **Follow-up:** *What if two actions run at once?* → Currently the second overwrites `pendingKey`. A `Set` of in-flight keys would be the fix; I haven't needed it because the UI disables other actions while one is busy.

Q13. How do you handle errors on the frontend? A. `request<T>` converts every failure into a typed error. A true network failure becomes `NetworkError`. A 401 clears the session. Everything else becomes an `ApiError` carrying the server's own `detail`, which the backend writes as a sentence for a person. Those surface as toasts. `extractDetail` handles the fact that FastAPI returns `detail` as a *string* for my `HTTPException`s and a *list* of field errors for validation failures. **Follow-up:** *Why not write frontend error messages?* → Then the same rule would be phrased in two places and drift. The server knows exactly why it refused; it should say so once.

Q14. What validation happens in the browser? A. Very little — required fields and basic types. The real validation is Pydantic on the server, and errors come back through `extractDetail`. The honest tradeoff: an invalid form costs a round trip, which is worse UX. What I get is exactly one source of validation truth, so client and server can't disagree. **Follow-up:** *How would you improve it?* → Mirror the cheap, stable rules — password length, required fields, deadline in the future — client-side for immediate feedback, while keeping the server authoritative. Never move a rule, only duplicate the obvious ones.

Q15. How do loading states work? A. Three levels: `AuthSplash` while the token is exchanged, `state.isLoading` for the initial domain load, and `useAction`'s `pendingKey` per action. A `DataGate` component handles the loading/error/empty/loaded branch so pages don't each reinvent it. **Follow-up:** *What about the case where data loads but is empty?* → That's a distinct branch with an `EmptyState` component — an empty list is a normal outcome, not an error.

Q16. Why does `useMatchAnalysis` prefer your own organisation's score? A. Because an NGO wants to know how *it* scores on a donation, not how the leader scores. It requests ten matches and picks its own entry if present, falling back to the top match. That also matters for honesty — showing a kitchen an 87 that belongs to a competitor would be misleading. **Follow-up:** *Why not compute the score client-side?* → The weights, the radius, and the reliability history all live on the server. A second implementation in the browser would drift from the one decisions are actually made on.

Backend (Q17–Q26)

Q17. Explain FastAPI's dependency injection as you use it. A. `Depends` lets me declare what an endpoint needs and have FastAPI supply it. `get_db` is a generator dependency — everything after `yield` is teardown, so the session always closes even on an exception. `get_current_user` depends on `get_db`, so dependencies compose, and FastAPI caches each one per request. `require_roles` is a dependency *factory*: it returns a closure that captured the permitted roles. **Follow-up:** *What's the payoff?* → In tests I replace the real database with `app.dependency_overrides[get_db]` and the application code never knows. That's the clearest argument for DI I can give.

Q18. How does `require_roles` work? A. It's a closure. `require_roles(UserRole.ngo, UserRole.admin)` returns a new function that has captured `roles`; that inner function depends on `get_current_user` and raises 403 if the role isn't permitted. The important application is at the router level: `APIRouter(prefix="/api/admin", dependencies=[Depends(require_roles(admin))])` gates every admin path in one line, so a new admin endpoint **cannot** be added unprotected by forgetting a decorator. **Follow-up:** *How would you add permissions finer than roles?* → A permission set on the user and a `require_permission("donation:verify")` dependency in the same shape. The call sites wouldn't change structurally.

Q19. Walk me through `update_status`. A. Load with relations or 404. Check the transition against `ALLOWED_TRANSITIONS[current]` → 409. Check the role against `TRANSITION_ROLES[target]`, with a narrow exception letting the owning donor cancel → 403. Then target-specific side effects: for `ACCEPTED`, resolve the recipient (an admin may name one, an NGO always resolves to its own, naming another is 403), refuse if unverified, bind it, increment `accepted_donations`, and freeze `match_score`. For `VOLUNTEER_ASSIGNED`, refuse if another courier already holds it. For `COMPLETED`, increment both completion counters. Finally append a `StatusEvent` with a server-stamped time and commit — all in one transaction. **Follow-up:** *Why is `COMPLETED` an NGO action rather than the volunteer's?* → The courier says "I delivered"; the recipient confirms "we received". The party with an incentive to overstate isn't the party who confirms.

Q20. Why a transition table instead of `if` statements? A. The rules are readable in one screen, testable without HTTP, and adding a state touches a known small set of places. With conditionals, "can a volunteer cancel?" is a question you answer by reading code instead of reading a table. **Follow-up:** *What happens if you add a status and forget the table?* → Two different failures. Forget `ALLOWED_TRANSITIONS` and you get a `KeyError` → 500. Forget `TRANSITION_ROLES` and `.get(target, set())` returns empty, so nobody can perform it. That fails closed, which is the right direction, but silently — I'd add a startup assertion that both dicts cover every enum member.

Q21. Why is `MATCHED` not an assignment? A. `MATCHED` records that the system has a suggestion; `recipient_id` is still null. Only `ACCEPTED` binds a recipient. Auto-assigning would mean the platform deciding who gets food without the kitchen agreeing — and a kitchen that can't actually collect it would silently block the donation. **Follow-up:** *Then why store `match_score` at creation?* → So the suggestion is visible immediately. It's re-frozen at acceptance to the score the actual decision was made on.

Q22. How do you prevent N+1 queries? A. The shared `_loaded` query eager-loads donor, recipient, volunteer→user, and events with `selectinload`. Without it, rendering 100 donations would be 1 query plus 4 per row — 401 queries. With it, 5 regardless of row count. **Follow-up:** *Why `selectinload` and not `joinedload`?* → `joinedload` on a one-to-many like `events` multiplies rows — a donation with six events comes back six times and SQLAlchemy has to de-duplicate. `selectinload` issues a second `SELECT ... WHERE id IN (...)`, which stays flat.

Q23. What happens if two volunteers claim the same pickup? A. The guard is `if donation.volunteer_id not in (None, volunteer.id): raise 409`. The second one gets "Another courier has already claimed this pickup", and there's a test for it. **But I'll be honest about the limit:** that's a read-then-write with no row lock and no unique constraint. SQLite serialises writes so it holds today, but on Postgres it's a genuine TOCTOU race — two requests could both read null and both write. **Follow-up:** *How would you fix it properly?* → Either `SELECT ... FOR UPDATE` on the donation row, or a conditional `UPDATE donations SET volunteer_id=? WHERE id=? AND volunteer_id IS NULL` and check the affected row count. The second is better — one atomic statement, no lock held across application logic.

Q24. Why is login form-encoded when everything else is JSON? A. It implements the OAuth2 password flow shape that FastAPI's `OAuth2PasswordRequestForm` expects, which is also what makes Swagger's "Authorize" button work. That's why `python-multipart` is a dependency. The field is called `username` and I treat it as an email. **Follow-up:** *Would you change it?* → Only if I dropped the Swagger convenience. The inconsistency is contained in one function on each side.

Q25. How are errors handled on the backend? A. Every error is an explicit `HTTPException` with a `detail` written as a sentence for a person — "This is the last active administrator, appoint another one first." FastAPI renders it as `{"detail": ...}` and the frontend shows it directly. There is **no custom exception handler and no global middleware**. **Follow-up:** *What's the gap?* → An unhandled exception returns a bare 500 with no body, and my frontend translates a bodiless 500 into "cannot reach the server" — so a crash looks like an outage. I'd add an exception handler that returns a correlation id and logs the traceback.

Q26. Why does the CLI exist? A. The API deliberately has no path to a first administrator, which leaves a bootstrap problem — the first one has to come from somewhere. It comes from whoever can already run commands against the database, which is the only authority that exists before any account does. It also prompts for passwords with `getpass` rather than taking arguments, because an argument lands in shell history and the process list. **Follow-up:** *Isn't that a backdoor?* → It grants nothing to anyone who didn't already have everything. If you can run `python -m foodlink.cli` against the production database, you could edit the rows directly anyway.

Database (Q27–Q35)

Q27. Walk me through your schema. A. Six tables. `users` holds every account with a `role` column — single-table inheritance, so authentication is one query regardless of role. `recipients` and `volunteers` are optional 1:1 satellites carrying role-specific data. `donations` is the central entity, referencing a donor and optionally a recipient and a volunteer. `status_events` is an append-only ledger of every transition. `requirements` holds standing needs so demand is visible before supply. **Follow-up:** *Why one users table rather than three?* → Authentication is one query against one index regardless of role, and adding a role is adding an enum value. The cost is nullable columns that don't apply to every row.

Q28. Why an append-only event table instead of timestamp columns? A. Four reasons. Metrics become queries over evidence rather than trust in columns. The trail records *who* acted, which a timestamp column can't express. Adding a state costs nothing, whereas each new timestamp would need a migration. And re-entering a state is representable — `MATCHED → AVAILABLE → MATCHED` is legal, and a single `matched_at` column would overwrite and lose the first attempt. **Follow-up:** *What's the cost?* → Reading "when was this accepted?" scans the donation's events instead of reading a column. Irrelevant at this scale; at large scale I'd denormalise the two or three timestamps that are read constantly.

Q29. Explain `UtcDateTime`. A. SQLite has no timezone type. `DateTime(timezone=True)` stores the naive wall clock and returns it with no offset, so the API serialises `2026-09-01T08:05:44` with no zone and the browser reads it as *local* time — a deadline four hours out appears ninety minutes past in IST. `UtcDateTime` is a `TypeDecorator` that normalises to UTC on write and reattaches UTC on read, so every datetime in Python is aware UTC. Postgres already behaves this way and passes through unchanged. **Follow-up:** *Why at the type layer?* → Because otherwise every query and every serialiser has to remember, and one that forgets produces a bug no unit test catches but that corrupts every deadline.

Q30. Why is `reliability_score` a property, not a column? A. It's derived from `accepted_donations` and `completed_donations`, so as a property it can never disagree with them. As a column it would need updating everywhere those counters change, and any missed path silently corrupts ranking. **Follow-up:** *Why 85 for newcomers?* → A cold-start hedge. An organisation with one lucky completion would sit at 100% and permanently outrank everyone, and a brand-new kitchen at 0% would never be matched and so could never earn a record. The threshold of 3 is judgement, not data — I'd tune it once there's history.

Q31. Why are recipient coordinates nullable but donation coordinates not? A. A donation without a location can't be matched or collected at all, so it's required. An NGO signs up before pinning its address, and forcing coordinates at registration turns a short signup into a survey. A recipient without coordinates simply isn't matchable — `score_pair` returns `None` — which is correct behaviour rather than an error. **Follow-up:** *How does the NGO fix it later?* → `PATCH /api/recipients/me`. That endpoint exists specifically to complete the profile.

Q32. What indexes do you have and why? A. `users.email` (unique — every login) and `users.role`. On donations: `donor_id`, `status`, `pickup_deadline`, `recipient_id`, `volunteer_id`, `created_at`. On `status_events`: `donation_id`, `to_status`, `occurred_at`. They match the actual access patterns — filter by status, order by deadline, scope by owner, fetch a donation's events. **Follow-up:** *Any index you'd add?* → A composite on `(status, pickup_deadline)` for the expiry sweep, which filters on both. And a spatial index on recipient coordinates once I push the radius filter into SQL.

Q33. How do you handle migrations? A. I don't, and that's my biggest infrastructure gap. Schema comes from `Base.metadata.create_all` at startup, which creates missing tables but does **not** alter existing ones. Add a column and the app starts fine, then fails on any query touching it. Today the workaround is deleting the database file. **Follow-up:** *What would you do?* → Add Alembic, autogenerate an initial revision from the current models, and replace `create_all` with `alembic upgrade head`. It must happen before any production deployment, because the first schema change after go-live would otherwise mean data loss.

Q34. Are your foreign keys enforced? A. They're declared, and on Postgres they'd be enforced. **On SQLite they are not**, because `PRAGMA foreign_keys = ON` is never issued and SQLite defaults to off. So on the default configuration they're documentation. That's a real finding I'd fix with a connection event listener. **Follow-up:** *What could go wrong?* → Orphaned rows — a `status_event` pointing at a deleted donation. In practice the ORM's cascades prevent it, but nothing outside the ORM is constrained.

Q35. Where are your invariants enforced? A. Almost entirely in the application: the state machine in Python dicts, `quantity > 0` and coordinate ranges in Pydantic, counter consistency in the router. The database enforces only uniqueness, nullability, and (on Postgres) foreign keys. **Follow-up:** *What's the risk?* → Anything writing to the database other than the API can violate every one of those rules. For a single-application database it's an acceptable trade for readability, but I'd push the cheap ones — CHECK constraints on quantity and capacity — down to the schema.

Authentication & Security (Q36–Q45)

Q36. Walk me through authentication end to end. A. Registration validates through Pydantic — including that the role is in `SELF_SIGNUP_ROLES`, so `admin` is rejected by the schema itself — hashes with bcrypt, inserts the user, flushes to get the id, creates the profile row for an NGO or volunteer, and commits. Login looks up by lowercased email, verifies with `bcrypt.checkpw`, checks `is_active`, and mints an HS256 JWT carrying `sub`, `role`, and a 12-hour `exp`. The token goes in localStorage. Every request sends it as a bearer header; `get_current_user` decodes it, **re-reads the user row**, and rejects if the account is gone or inactive. `require_roles` then gates by role. **Follow-up:** *Why re-read the user when the token has the role?* → So an administrator suspending an account takes effect immediately rather than whenever the token expires. It costs one indexed primary-key lookup per request. There's a test named exactly that: `test_a_suspended_accounts_existing_token_stops_working`.

Q37. Why bcrypt? A. SHA-256 is designed to be fast, which is precisely wrong for passwords — a GPU does billions per second. bcrypt is deliberately slow with a tunable cost factor, and `gensalt()` produces a unique salt per password, stored inside the hash. So two users with the same password get different hashes and a rainbow table is useless. **Follow-up:** *Any gotcha?* → bcrypt silently truncates at 72 bytes while my schema allows 128 characters. Not exploitable at realistic lengths, but it means the tail of a very long passphrase is ignored. Argon2id would be the modern choice.

Q38. What's in your JWT, and is it safe to put a role in it? A. `sub` (the user id as a string, per spec), `role`, and `exp`. A JWT is **signed, not encrypted** — anyone can decode and read it — so it must never carry a secret. An id and a role are not secrets. And the `role` claim is effectively decorative: `get_current_user` ignores it and uses the live database row, so a stale claim can't grant anything. **Follow-up:** *Then why include it?* → Convenience for debugging and a possible future fast path. Given it's never trusted, I could drop it.

Q39. How do you log out, and what are the implications? A. `signOut()` clears the token from localStorage and the user from state. **It's client-side only — there's no server call and no blocklist**, so a token copied before logout stays valid until it expires. That's the standard JWT tradeoff and I should state it plainly. **Follow-up:** *How would you get real revocation?* → The cheapest option is a `token_version` integer on `users`, included in the token and compared in `get_current_user`; bumping it invalidates every existing token for that account. A full blocklist in Redis is the heavier alternative. I'd also shorten the access token to about 15 minutes and add refresh tokens.

Q40. Why is the token in localStorage rather than an httpOnly cookie? A. The session survives a reload, there's no CSRF surface because a custom header isn't attached automatically by the browser, and the API stays stateless and horizontally scalable. The cost is that any script on the page can read it, so an XSS bug becomes account takeover. The code documents this trade honestly.

Follow-up: *So how do you handle CSRF?* → The auth scheme makes it inapplicable — browsers attach cookies to cross-site requests automatically, but not `Authorization` headers, so the classic attack has nothing to ride on. If I moved to cookie auth, CSRF tokens and `SameSite` would immediately become mandatory. **That's the answer — not "we don't do CSRF".**

Q41. Why can't someone register as an admin? A. `SELF_SIGNUP_ROLES` is a frozenset of donor, ngo, and volunteer, and a Pydantic `field_validator` on `RegisterRequest` rejects anything else. It's in the schema rather than the router deliberately, so the restriction appears in the published OpenAPI document — the contract itself says `admin` isn't an accepted value. Otherwise "become an admin" would be one unauthenticated POST. **Follow-up:** *So how does the first admin exist?* → `python -m foodlink.cli create-admin`, run by whoever controls the database. After that, admins create admins through `POST /api/admin/users`.

Q42. What stops an NGO accepting a donation on another kitchen's behalf? A. For a non-admin, the recipient is always resolved from `SELECT recipients WHERE user_id = me` — never from the request body. If the body names a different `recipientId`, that's an explicit 403. Only an administrator may name an arbitrary organisation, because acting for someone else is a support action, not something a peer does to a competitor for the same donation. Both paths are tested. **Follow-up:** *Why let admins do it at all?* → Support. A kitchen phones in because their account is broken and someone has to accept on their behalf.

Q43. What is `is_verified` and why two enforcement points? A. It's an administrator vouching that a real organisation stands behind the account. It's enforced in two places: `score_pair` returns `None` for unverified organisations so they never appear in rankings, and `update_status` refuses `ACCEPTED` with a 403. Both are necessary — ranking alone is a UI filter a client could bypass; the accept check is the actual control. There are separate tests for each half. **Follow-up:** *What happens if verification is revoked mid-lifecycle?* → Honestly, nothing — the endpoint just flips the flag. An already-accepted donation continues. That's an untested edge case and I'd want to decide deliberately whether it should be reassigned.

Q44. What's the biggest security weakness in your project? A. Two, and I'd fix them in this order. First, the signing key falls back to a **default published in the repository** if `FOODLINK_SECRET_KEY` isn't set, and the app starts anyway — so a misconfigured deployment lets anyone forge an admin token. The fix is four lines: refuse to boot on the default outside development. Second, `GET /api/donations` defaults to `mine=false` and `GET /api/donations/{id}` has no ownership check, so **any authenticated account can read every donation**, including exact coordinates and donor names. Some breadth is required — an NGO must browse available donations — but completed and other organisations' accepted donations shouldn't be world-readable. **Follow-up:** *Why didn't you fix them?* → Scope and time; the project was built to demonstrate the coordination model. But I know exactly where they are and what the fix costs, and I'd rather tell you than have you find them.

Q45. Are you vulnerable to SQL injection or XSS? A. SQL injection: structurally no. Every query is SQLAlchemy's expression language, which parameterises. There's no raw SQL, no `text()`, and no f-string-built query anywhere in the backend — user data never reaches the SQL parser as code. XSS: largely no. React escapes everything rendered as `{value}`, and `dangerouslySetInnerHTML` appears nowhere. The residual risk is that `image_url` is an unvalidated text field, and that my token is in localStorage, so any XSS I did introduce would be more costly than it needs to be. **Follow-up:** *How would you reduce that?* → A Content-Security-Policy header, which the app currently doesn't send at all.

Matching, APIs & Testing (Q46–Q52)

Q46. Explain the matching algorithm. A. For each recipient, three hard gates first: unverified, missing coordinates, or beyond the 8 km radius returns `None` — gating rather than scoring, because a suggestion the recipient couldn't act on would be a false promise. Survivors get five normalised 0–100 scores: distance (linear decay to the radius), quantity fit (peaks at exactly 100% of capacity, with overflow penalised at twice the slope of underfill), remaining capacity, deadline slack after subtracting travel at an assumed 20 km/h, and reliability. Weighted 0.25/0.25/0.20/0.15/0.15 — summing to exactly 1.0 — then sorted descending. **Follow-up:** *Compute one for me.* → 120 meals, 1.2 km away, capacity 150, plenty of time, reliability 92: distance 85, quantity 88, capacity 60, deadline 100, reliability 92 → 21.25 + 22 + 12 + 15 + 13.8 = **84**.

Q47. Is this AI? A. No, and I want to be precise about that. It's a transparent weighted sum over five criteria — an explainable heuristic, not a learned model. That was deliberate: there's no training data before the platform is used, a recipient needs to see *why* they were ranked, and a marker can verify it by hand. The code is structured so a learned ranker would replace `score_pair` alone; the router and response shape wouldn't change. **Follow-up:** *What would you need to make it ML?* → Labelled outcomes — which matches were accepted, which completed on time. Then learning-to-rank over the same features, with the heuristic as the baseline to beat. I'd also keep the explanation, because "the model said so" is not something you can tell a kitchen.

Q48. Why does the score get recalculated at acceptance? A. To freeze the number the decision was actually made on. Capacity and reliability change over time, so a score computed live at display time would drift away from what justified the acceptance. Storing it at that moment makes the record honest. **Follow-up:** *Why store it at creation too?* → So the donation carries a suggestion the moment it's posted. That one is provisional; the acceptance one is the record.

Q49. Design a new endpoint: an NGO rejecting a donation. Walk me through it. A. Add `REJECTED` to `DonationStatus`; add it to `ALLOWED_TRANSITIONS[ACCEPTED]` and give it its own entry (probably `set()`, or back to `AVAILABLE` so it can be re-offered); add `TRANSITION_ROLES[REJECTED] = {ngo, admin}`; in `update_status`, clear `recipient_id` and decrement `accepted_donations` so reliability isn't unfairly penalised; add the literal to the TypeScript union and handle it in `StatusBadge`. Six small places — that's the payoff of the table-driven design. **Follow-up:** *Should rejection hurt reliability?* → No, and that's the interesting product question. Reliability measures whether accepted donations get completed. Punishing an honest early rejection would push kitchens to accept

and then fail, which is strictly worse for the food.

Q50. How do you test the backend? A. 37 pytest tests through FastAPI's `TestClient`, all going through the full stack — routing, validation, DI, authorization, ORM, and a real in-memory SQLite database. **No mocks anywhere.** `conftest.py` creates the database with `StaticPool`, which is the key trick: in-memory SQLite exists per connection, so the default pool would hand out a new connection and the tables would vanish. Then `app.dependency_overrides[get_db]` swaps the session in without the application knowing. **Follow-up:** *Integration or unit?* → All 37 are integration tests. There are **zero unit tests**, which is a gap — `matching.py` is pure and trivially unit-testable but is only exercised through the API, so the exact curve of the quantity score at the overflow boundary is unverified in isolation.

Q51. What are you not testing? A. The entire frontend — 83 TypeScript files, zero tests of any kind; `tsc` in the build is the only gate. The `UtcDateTime` decorator, which is ironic given it exists to prevent a subtle bug. Real concurrency — the courier race is tested sequentially, which doesn't exercise the actual window. And most 422 branches. **Follow-up:** *What would you write first?* → Unit tests for the five scoring functions at their boundaries, because that's where a silent behaviour change would be invisible today. Then a round-trip test for `UtcDateTime`.

Q52. Do your tests run in CI? A. No. The only workflow is `.github/workflows/mkdocs.yml`, which deploys documentation. A commit breaking all 37 tests merges to master with no signal. That's the highest-value process fix available and it's about twenty lines of YAML — `pytest` on the backend and `npm run build` on the frontend, since the build runs `tsc` and would catch type regressions.

Deployment, Performance & Design (Q53–Q56)

Q53. How would you deploy this? A. It isn't deployed and there's **no deployment configuration at all** — no Docker, no Procfile, no CI beyond docs. What it would need: generate and set `FOODLINK_SECRET_KEY`; move to Postgres via `DATABASE_URL` — but **add Alembic first**, because shipping `create_all` means the first schema change loses data; set `CORS_ORIGINS` to the real frontend origin; run uvicorn with several workers behind nginx terminating TLS; build the frontend and serve `dist/` statically **with an SPA rewrite to `index.html`**, or deep links 404 on refresh; bootstrap an admin with the CLI; and add a scheduler for the expiry sweep. **Follow-up:** *Why can't you run multiple workers today?* → SQLite has a single writer. Concurrent workers would produce lock errors and risk corruption. Multiple workers requires Postgres first.

Q54. What breaks first at 1,000 users? A. SQLite write contention — "database is locked" during busy periods. Then `GET /api/metrics`, which loads **every** donation and **every** status event into memory and computes medians in Python; at 50,000 donations that's hundreds of thousands of rows per dashboard load. Then the lack of pagination — there's a `limit` capped at 500 but no `offset` or cursor, so lists are simply truncated. **Follow-up:** *Fix the metrics endpoint.* → Push the aggregation into SQL — counts and `percentile_cont` for medians on Postgres — and cache the result for sixty seconds. At real scale, a rollup table refreshed on a schedule, because platform metrics don't need to be second-accurate.

Q55. Your config says the radius bounds the ranking work. Does it? A. No, and that's a good catch. `rank_recipients` is called with `select(Recipient)` — every organisation, with no `WHERE` clause. The radius filter happens in Python *after* every row is loaded, so the cost is O(all organisations) per donation posted. The comment describes the intent, not the implementation. **Follow-up:** *Fix it.* → A bounding-box predicate on latitude and longitude in SQL — cheap, uses an index, needs no extension — then exact haversine on the survivors. At real scale, PostGIS with a GiST index and `ST_DWithin`.

Q56. If you rebuilt this, what would you change? A. Five things, in order. Postgres and Alembic from day one — the migration gap is the one decision that would actually hurt in production. CI running the tests from the first commit. Scope the donation reads by role properly instead of relying on a `mine` flag nobody has to pass. Generate the TypeScript client from the OpenAPI schema instead of hand-mirroring the types. And use React Query for server state, which would fix the chattiness of write-then-refetch without giving up correctness. **What I'd keep:** the append-only event ledger, the table-driven state machine, the explainable matcher, and re-reading the user on every request. Those four are the decisions I'd defend anywhere.

---

20. Explain My Project in an Interview

30 seconds

> "FoodLink is a platform that connects restaurants with surplus food to nearby community kitchens before that food expires. A donor posts what they have with a location and a deadline; the server ranks nearby verified kitchens and suggests the best match; a kitchen accepts, a volunteer collects and delivers. Every step is timestamped server-side, so the platform can actually prove how fast it moved food. It's a React frontend and a FastAPI backend with SQLAlchemy."

1 minute

> "The problem is that surplus food has a deadline measured in hours, and the slow part isn't transport — it's finding someone who can take it in time.

> FoodLink is a coordination platform for that. A donor posts food with coordinates and a pickup deadline. The server immediately ranks every verified kitchen within an eight-kilometre radius on five criteria — distance, how well the quantity fits their capacity, remaining capacity, whether the deadline is comfortably reachable, and their completion record — and attaches a match score with the reasoning. A kitchen accepts, a volunteer claims the pickup, collects, delivers, and the kitchen confirms receipt.

> The part I'm most pleased with is that every transition is written to an append-only event table with a server-stamped time and the user who did it. That means the metrics — median time-to-claim, rescue rate, expiry loss — are derived from evidence rather than self-reported. It's React and TypeScript on the front, FastAPI, SQLAlchemy and Pydantic on the back, with JWT auth and four roles."

3 minutes — technical

> "Let me start with the domain, because the architecture follows from it.

> A donation moves through nine states — available, matched, accepted, volunteer-assigned, picked-up, delivered, completed, plus cancelled and expired. The legal transitions are a dictionary in the models module, and a second dictionary maps each target state to the roles allowed to cause it. So the whole lifecycle is data you can read in one screen, not conditionals scattered through a handler. An illegal transition is a 409, a wrong role is a 403.

> Crucially, transitions aren't stored as timestamp columns. Every one appends a row to a `status_events` table — from-state, to-state, actor, and a time stamped by the server, never accepted from the client. That's what makes the metrics honest: time-to-claim is the median gap between creation and the accepted event, and rescue rate is the share of donations completed before their stated deadline. It also means adding a new state doesn't need a migration, and re-entering a state is representable.

> Matching is a weighted sum over five normalised criteria, and I want to be clear that it's an explainable heuristic, not machine learning. There's no training data before the platform is used, and a kitchen deserves to see why it was ranked — so the endpoint returns all five sub-scores and human-readable reasons alongside the total. Unverified organisations and ones outside the radius are gated out entirely rather than scored low.

> On auth: bcrypt for passwords, HS256 JWTs as bearer tokens. The decision I'd highlight is that the token carries a role, but the server ignores it — every request re-reads the user row and checks `is_active`. That's one indexed lookup, and it buys immediate suspension mid-session rather than waiting for a token to expire. There's a test named exactly that.

> Authorization is four layers: role, ownership, lifecycle legality, and trust — an organisation has to be verified by an admin before it can accept anything, and that's enforced both in the ranking and at the transition.

> On the frontend, two React Contexts — identity separate from domain data, because identity has to settle before we know what we're allowed to fetch — and one module that owns every HTTP call, so token attachment, error normalisation, and global session expiry are solved once."

5 minutes — deep technical

Deliver the 3-minute version, then continue:

> "A few implementation details worth calling out.

> There's a custom SQLAlchemy `TypeDecorator` called `UtcDateTime`. SQLite has no timezone type — it stores the naive wall clock and hands it back with no offset, so the API would serialise a time with no zone and the browser would read it as local. A deadline four hours out would display as ninety minutes past in IST. The decorator normalises to UTC on write and reattaches it on read, so every datetime in Python is aware UTC. Postgres already behaves that way and passes through unchanged. It's the kind of bug no unit test catches but that would corrupt every deadline in the system.

> On query performance: the donation queries eager-load donor, recipient, volunteer and events with `selectinload`. Without it, a hundred-row list is 401 queries; with it, five. I used `selectinload` rather than `joinedload` specifically because a join on the one-to-many events would multiply rows.

> On testing: 37 integration tests, no mocks, all through the real stack against an in-memory SQLite database. The trick there is `StaticPool` — in-memory SQLite exists per connection, so the default pool would give the test and the request different databases. Then `dependency_overrides` swaps the session in without the application knowing, which is the clearest argument for dependency injection I can give.

> Now, the honest weaknesses, because I'd rather raise them than have you find them. There are no migrations — schema comes from `create_all`, which creates missing tables but doesn't alter existing ones, so the first schema change in production would lose data. That's the first thing I'd fix. The signing key has a default that's published in the repo and the app starts anyway if the environment variable is missing. Donation reads are broader than they should be — any authenticated user can list every donation, including coordinates. There's no rate limiting, so login is brute-forceable. And the tests don't run in CI at all; the only workflow deploys documentation.

> There's also a concurrency caveat I'd flag: the guard stopping two couriers claiming the same pickup is a read-then-write with no row lock. SQLite serialises writes so it holds today, but on Postgres it's a real race. The fix is a conditional update with a row-count check rather than a check-then-set.

> What I'd defend without hesitation: the event ledger, the table-driven state machine, the explainable matcher, and re-reading the user on every request."

---

21. Project Defense

Hard questions, with honest answers.

"Why SQLite? That's not a real database."

> It's a real database — it's the most deployed one in the world — but it's the wrong one for a multi-writer production service, and I'd agree with that. I chose it so a fresh clone runs with zero setup, which matters when a marker has five minutes. The important part is that I didn't paint myself into it: everything goes through SQLAlchemy, and switching to Postgres is one environment variable. What I'd have to do *first* is add Alembic, because I currently have no migrations.

"Why not a NoSQL database?"

> The data is deeply relational — users to organisations to donations to events — and the core value of the system is derived from joins and aggregates over the event table. Losing joins and transactions to gain schema flexibility I don't need would be a bad trade. The one place a document store would fit is the event log itself, if it grew beyond what a relational table handles comfortably.

"What happens if the database goes down?"

> Every request that touches it returns a 500, which the frontend currently reports as "cannot reach the server" — misleading, and something I'd fix. `/api/health` would still return OK, because it doesn't touch the database at all, so a naive health check would report the service as healthy while every request failed. That's a real gap: a readiness probe should run `SELECT 1`. `pool_pre_ping=True` is set, so a stale connection is detected rather than hanging, but there's no retry, no circuit breaker, and no degraded read-only mode.

"What happens if two users modify the same donation at once?"

> Two cases. For most transitions, `ALLOWED_TRANSITIONS` protects me: if two people both try to accept, the first succeeds and the second finds the status is no longer acceptable and gets a 409. For the courier claim I have an explicit guard and a test. **But I'll be honest that the guard is a read-then-write with no row lock.** SQLite serialises writes so it holds today, but on Postgres two requests could both read `volunteer_id IS NULL` and both write. The proper fix is a single conditional update — `WHERE volunteer_id IS NULL` — and checking the affected row count, so the atomicity is the database's job rather than my application's.

"Your matching is just a weighted average. Where's the AI?"

> There isn't any, and I'd rather say that than oversell it. It's an explainable heuristic, and that was deliberate: there's no labelled data before the platform is used, a kitchen deserves to see why it was ranked, and it can be verified by hand. What I did do is structure it so the swap is contained — replacing `score_pair` gets you a learned ranker without touching the router or the response shape. To actually train one I'd need acceptance and completion outcomes, and I'd keep the explanation regardless, because "the model said so" isn't something you can tell a community kitchen.

"How do you know your weights are right?"

> I don't. They're a judgement call — distance and quantity fit at 0.25 each because they're the two that most often decide whether a handover is possible at all, then capacity, then deadline and reliability. There's no evidence behind those numbers, and the honest answer is that they're a starting point to be tuned against outcomes once the platform has history. Same for the 20 km/h travel assumption and the 85 reliability prior.

"What's the weakest part of your system?"

> The absence of migrations. Everything else on my list is a bounded fix — rate limiting is a library, the secret key is four lines, scoping the reads is a `WHERE` clause. But `create_all` doesn't alter existing tables, so the first schema change after a production deployment either loses data or requires a manual `ALTER` I'd have to get exactly right with no rollback. It's the one gap that gets more expensive the longer it's left.

"An attacker gets a valid token. What can they do, and how do you stop them?"

> They act as that user for up to twelve hours. There's no blocklist, so I can't revoke a specific token. What I *can* do is suspend the account — and that works immediately, because `get_current_user` re-reads `is_active` on every request rather than trusting the token. So the containment path exists, it's just coarse: I can kill the account, not the session. Proper revocation would be a `token_version` column compared on each request, plus much shorter access tokens with refresh.

"Can a volunteer see donations they're not assigned to?"

> Yes — and more than they need to. `GET /api/donations` defaults to `mine=false`, so any authenticated account sees every donation including exact coordinates and donor names, and `GET /api/donations/{id}` has no ownership check. Some breadth is necessary, because a courier has to see pickups available to claim. But completed donations and other organisations' accepted ones shouldn't be readable. The fix is scoping the default listing by role, and it's the authorization gap I'd fix first.

"Why should I trust your metrics?"

> Because none of them are self-reported. Every number comes from `status_events`, and `occurred_at` is stamped by the server at the moment of the transition — the column is never populated from a request body. A donor can't claim they posted earlier and a kitchen can't claim it collected faster. The limitation I'd name is that the expiry sweep has no scheduler, so unclaimed donations sit as available until someone calls the endpoint — which means the expiry-loss rate currently *understates* reality.

"What if an admin account is compromised?"

> That's total compromise — an admin can create accounts of any role, verify organisations, suspend anyone, and drive any transition. The mitigations that exist are narrow: an admin can't suspend or demote themselves, and the last active admin can't be removed, so the platform can't be locked out. But there's no audit log of admin actions beyond what lands in `status_events`, no MFA, and no approval flow for privileged operations. For a real deployment I'd add MFA for admins first, then an audit table.

"What would you change if you rebuilt it?"

> Postgres and Alembic from the first commit, and CI running the tests from the first commit — those two are pure process and cost nothing early, but are painful to retrofit. Then I'd generate the TypeScript client from the OpenAPI schema rather than hand-mirroring types. And I'd use React Query for server state, which would keep the correctness of write-then-refetch while removing the chattiness. What I'd keep unchanged: the event ledger, the state machine as data, the explainable matcher, and re-reading the user every request.

"How much of this did you actually write?"

> I used AI assistance substantially while building it, and I'd rather say that than pretend otherwise. What I can do is defend every decision in it — why events instead of timestamp columns, why the user is re-read on every request rather than trusting the token's role claim, why `selectinload` and not `joinedload`, why `StaticPool` is required for the in-memory test database, and where the code is wrong. Ask me to change anything in it and I'll tell you which files and what breaks.

---

22. Improvement Roadmap

CRITICAL — before any production use

#	Item	Why	Effort

1	**Add Alembic**	`create_all` can't alter tables; the first schema change loses data	~2 h
2	**Fail fast on the default secret key**	A misconfigured deploy allows forged admin tokens	~15 min
3	**Scope donation reads by role**	Any account can read every donation incl. coordinates	~2 h
4	**Move to Postgres for deployment**	SQLite's single writer blocks multiple workers	~1 h + Alembic

HIGH

#	Item	Why	Effort
5	**CI running pytest + `npm run build`**	37 tests exist and never run automatically	~30 min
6	**Rate-limit auth endpoints**	Login is brute-forceable	~1 h
7	**Fix the courier claim race**	Read-then-write TOCTOU on Postgres	~1 h
8	**Schedule the expiry sweep**	Metrics understate expiry loss until someone calls it	~1 h
9	**Pagination (cursor or offset)**	Lists silently truncate at 500	~3 h
10	**Push the match radius into SQL**	Ranking loads every organisation; the stated bound isn't real	~2 h
11	**Shorten tokens + add refresh or `token_version`**	12 h tokens, no revocation	~4 h

MEDIUM

#	Item	Why
12	Aggregate `/api/metrics` in SQL + cache	Full table scan per dashboard load
13	Unit tests for `matching.py`	Pure functions, boundaries currently unverified
14	Frontend tests (Vitest + Testing Library)	83 files, zero tests
15	`PRAGMA foreign_keys=ON` for SQLite	FKs currently unenforced
16	`PATCH`/`DELETE` for requirements	Can be created and listed, never edited or deactivated
17	Structured logging	No `logging` configuration at all
18	Global exception handler + correlation ids	A 500 currently looks like an outage to the client
19	Real image upload to object storage	Base64 in a text column inflates every response
20	Readiness probe that checks the DB	`/api/health` reports healthy with a dead database

21	Security headers + CSP	Nothing is sent today
22	Remove or wire up dead code: `COLD_STORAGE`, `useIsMobile`, `Volunteer.rating`	Defined but never used / never written
23	Delete the stale C++ Makefile, `src/`, `inc/`, `run_main.o`	Template residue that confuses readers
24	Fix `pyproject.toml`	Still names the template; `requires-python` says 3.8 but the code needs 3.10+

LOW / future features

#	Item
25	Generate the TS client from OpenAPI
26	React Query for server state
27	Code splitting per portal (`React.lazy`)
28	Email/SMS notifications on match and assignment
29	WebSockets for a live donation feed
30	Real routing distance instead of haversine
31	Tune weights against outcome data
32	Recipient food-category preferences (and actually use `COLD_STORAGE`)
33	Courier rating flow (the column exists and is never written)
34	MFA + an audit log for admin actions
35	Recurring donation schedules

Known bugs and questionable behaviour

1. **A donation accepted but never delivered is stuck forever.** The expiry sweep only touches `AVAILABLE` and `MATCHED`. Deliberate or an oversight? It should be decided explicitly.
2. **`COLD_STORAGE` is defined and never referenced.** The comment says storage type should gate matching; it doesn't.
3. **`Volunteer.rating` is never written by any API path.** Every courier created through the app is permanently 5.0; only `seed.py` varies it.
4. **`useIsMobile.ts` is dead code** — never imported. Nothing routes a phone visitor to the `/m/*` mobile UI automatically.
5. **Revoking verification mid-lifecycle has no effect** on an already-accepted donation. Untested and undecided.
6. **Two `foodlink.db` files exist** because the SQLite path is relative to the working directory — a recurring "my data vanished" trap.
7. **`image_url` has no length or format validation.**
8. **A 500 is reported to the user as a network failure.**

---

23. Learning Roadmap

Based only on what this project actually uses.

Stage 1 — Solidify what you already touched (1–2 weeks)

1. **HTTP and REST fundamentals** — status codes, idempotency, headers vs body. *Why:* you'll be asked 401 vs 403 and 409 vs 422, and this project models them correctly. Know why `POST /status` isn't pure REST.
2. **SQL by hand** — write the joins and aggregates `/api/metrics` performs, as raw SQL. *Why:* you can't defend an ORM you can't out-write. Start with: "median minutes from creation to the ACCEPTED event, per month."
3. **JWT deeply** — structure, signing vs encryption, `alg: none`, HS256 vs RS256, why `algorithms=[...]` must be pinned. *Why:* the most-probed area in any interview touching auth.

Stage 2 — Close this project's real gaps (2–4 weeks)

4. **Alembic** — autogenerate a revision from the current models and actually run an upgrade and a downgrade. *Why:* it's the project's biggest gap; having fixed it is a far better story than knowing about it.
5. **PostgreSQL** — run this app against it. Then `EXPLAIN ANALYZE` the donation list query and the metrics query. *Why:* turns "I could switch" into "I did, and here's what changed."
6. **SQLAlchemy beyond the basics** — sessions, identity map, `flush` vs `commit`, lazy vs eager loading, `selectinload` vs `joinedload`, and `SELECT ... FOR UPDATE`. *Why:* directly answers your N+1 and race questions.
7. **pytest properly** — fixtures, parametrisation, and writing unit tests for `matching.py`'s boundaries. *Why:* your weakest coverage area.

Stage 3 — Frontend depth (2–3 weeks)

8. **React rendering and hooks internals** — dependency arrays, `useRef` vs `useState`, effect cleanup, why Context re-renders every consumer. *Why:* your Context choice needs a defence, and the 401 guard needs explaining.
9. **React Query** — refactor one screen off write-then-refetch. *Why:* it's the change you say you'd make; making it is much stronger.
10. **TypeScript generics and utility types** — `Partial`, `Omit`, `Exclude`, generic functions. *Why:* your codebase already uses `Exclude<UserRole,'admin'>` to encode a security rule in the type system.
11. **Vitest + Testing Library** — write the first frontend test for `ProtectedRoute`. *Why:* zero frontend tests is the gap most likely to be challenged.

Stage 4 — Production skills (3–4 weeks)

12. **Docker** — containerise the backend, then compose it with Postgres. *Why:* the single most expected skill this project lacks.
13. **GitHub Actions** — the twenty lines of CI in §14.5. *Why:* cheap, and "the tests don't run" is otherwise an easy hit.
14. **nginx / reverse proxies** — TLS termination and the SPA rewrite rule. *Why:* explains the deep-link 404 you'd otherwise hit in production.
15. **Web security in practice** — OWASP Top 10, CSP, rate limiting with `slowapi`. *Why:* you have a specific weakness list; learn the specific fixes.

Stage 5 — Scale (ongoing)

16. **Caching with Redis** — cache `/api/metrics`.
17. **Background jobs** — APScheduler or Celery for the expiry sweep.
18. **PostGIS** — proper spatial indexing for the radius filter.
19. **Observability** — structured logging, then metrics and tracing.

Deliberately not on this list: Kubernetes, microservices, GraphQL, Kafka, system design at FAANG scale. None of them appear in this project, and claiming them invites questions you can't ground in anything you built.

---

24. One-Page Cheat Sheet

Stack

Frontend React 18.3 · TypeScript 5.5 · Vite 5.4 · React Router 6.26 · Tailwind 3.4 · lucide-react — *no Redux, no Axios, no form library, no tests* **Backend** FastAPI ≥0.115 · SQLAlchemy 2.0 · Pydantic 2.9 · PyJWT · bcrypt · uvicorn **DB** SQLite default → Postgres via `DATABASE_URL` **Tests** pytest — **37 integration tests, all passing, no mocks, no CI External services: NONE**

Architecture

`React SPA → lib/api.ts → (Vite proxy in dev) → FastAPI → SQLAlchemy → SQLite` Two Contexts (Auth = identity, App = domain). Five routers. No service layer.

Database — 6 tables

`users` (all roles, single table) · `recipients` · `volunteers` · `donations` · **`status_events` (append-only, server-stamped — the key design)** · `requirements` No migrations. No many-to-many. FKs unenforced on SQLite.

Lifecycle — 9 states

`AVAILABLE → MATCHED → ACCEPTED → VOLUNTEER_ASSIGNED → PICKED_UP → DELIVERED → COMPLETED` plus `CANCELLED` / `EXPIRED` Rules are **data**: `ALLOWED_TRANSITIONS` (models.py) + `TRANSITION_ROLES` (donations.py). Illegal → **409**. Wrong role → **403**. **`MATCHED` assigns nobody — only `ACCEPTED` binds a recipient.**

Auth

bcrypt (per-password salt) → HS256 JWT `{sub, role, exp:+720min}` → localStorage `foodlink.token` → `Authorization: Bearer` **`get_current_user` re-reads the user row every request** → instant suspension. The token's `role` claim is never trusted. `admin` cannot self-register (`SELF_SIGNUP_ROLES`) → bootstrap via `python -m foodlink.cli create-admin`. No refresh tokens. No revocation. Logout is client-side only.

Authorization — 4 layers

Role (`require_roles`) → Ownership → Lifecycle (`ALLOWED_TRANSITIONS`) → Trust (`is_verified`)

Matching — heuristic, NOT ML

distance **.25** · quantity fit **.25** · capacity **.20** · deadline **.15** · reliability **.15** (= 1.0) Hard gates: unverified → excluded · no coordinates → excluded · >8 km → excluded Reliability = 85 if <3 accepted, else `100 × completed/accepted`

Key APIs

`POST /api/auth/login` **(form-encoded!)** · `POST /api/auth/register` `POST /api/donations` (auto-ranks on create) · `GET /api/donations?mine=&status=` **`POST /api/donations/{id}/status`** ← the core endpoint `GET /api/donations/{id}/matches` (scores + reasons) `GET /api/metrics` · `POST /api/admin/recipients/{id}/verify` `POST /api/admin/maintenance/expire` · `GET /api/health` · docs at **`/docs`**

Key files

`models.py` (tables + lifecycle) · `security.py` (74 lines, all of auth) · `matching.py` (the scorer) · `routers/donations.py` (`update_status`) · `lib/api.ts` (the only `fetch`) · `AuthContext.tsx` · `conftest.py` (`StaticPool`)

Design decisions to lead with

1. Append-only `status_events` → honest, derived metrics
2. Table-driven state machine → readable, testable, extensible
3. Re-read the user every request → immediate suspension
4. Explainable heuristic, not ML → no training data, kitchens deserve reasons
5. `UtcDateTime` decorator → prevents a silent timezone corruption
6. `selectinload` → 401 queries become 5

Security

No SQL injection (ORM only, no raw SQL) · No XSS vector (no `dangerouslySetInnerHTML`) · CSRF N/A (header, not cookie) · bcrypt · No account enumeration at login · CORS allowlist · Admin router gated once **Default secret key, app starts anyway** · **Any user can read all donations** · No rate limiting · Token in localStorage · No revocation · SQLite FKs off

Deployment

None configured. No Docker, no CI for tests (only mkdocs). Would need: `FOODLINK_SECRET_KEY`, Postgres + **Alembic first**, `CORS_ORIGINS`, uvicorn workers behind nginx/TLS, static `dist/` **with an SPA rewrite**, CLI bootstrap, a scheduler for expiry. Env: `DATABASE_URL` · `FOODLINK_SECRET_KEY` · `ACCESS_TOKEN_MINUTES=720` · `CORS_ORIGINS` · `MAX_MATCH_RADIUS_KM=8`

Biggest limitations (say these before you're asked)

1. No migrations — `create_all` can't alter tables
2. Default signing key, no fail-fast
3. Donation reads not scoped by ownership
4. Metrics scan the whole table into memory
5. Ranking loads every organisation (the radius doesn't bound SQL)
6. No rate limiting · no CI · no frontend tests
7. Courier claim is a read-then-write race
8. SQLite = one writer = no multi-worker deployment

Terminology

Donor / Recipient (NGO) / Volunteer / Admin — the four roles **Match score** — 0–100 weighted sum, frozen at acceptance **Reliability score** — completion rate, 85 prior under 3 donations **Status event** — one append-only, server-stamped transition row **Time-to-claim** — median minutes, creation → `ACCEPTED` (the primary metric) **Rescue rate** — completed before deadline ÷ (completed + expired) **Verification** — an admin vouching; gates both ranking and acceptance **Terminal state** — `COMPLETED` / `CANCELLED` / `EXPIRED` (empty transition set)

If You Only Have 2 Hours to Study This Project

Follow this order. It is built so that stopping early still leaves you able to hold a conversation.

■ 0:00–0:20 — The domain and the lifecycle (highest value per minute)

Open `code/foodlink/models.py`. - Read `DonationStatus` and **`ALLOWED_TRANSITIONS`**. Draw the state diagram on paper. Note the three terminal states with empty sets. - Read the `StatusEvent` class and its docstring. **Be able to say in one sentence why history is a table and not five timestamp columns** (§6.2). - Read `Recipient.reliability_score` and know why the prior is 85. - Skim `UtcDateTime` — know *what problem it solves*, not the implementation.

You can now explain what the system does and its best design decision.

■ 0:20–0:45 — Authentication (the most-asked area)

Open `code/foodlink/security.py` — only 74 lines, read all of it. - `hash_password` / `verify_password` — why bcrypt, not SHA-256. - `create_access_token` — the three claims, 12-hour expiry. - **`get_current_user` — and why it re-reads the user row instead of trusting the token's `role`.** This is the single highest-value fact in the project. - `require_roles` — a closure returning a dependency.

Then `SELF_SIGNUP_ROLES` in `models.py` and the validator in `schemas.py`, plus the docstring of `cli.py` — the two-tier admin model.

You can now answer the auth questions, which are most of any interview.

■ 0:45–1:10 — The core endpoint

Open `code/foodlink/routers/donations.py`. - `TRANSITION_ROLES` — memorise which role drives which transition, especially that **`COMPLETED` is the NGO's, not the volunteer's**, and why. - Read **`update_status`** line by line. Trace the five checks in order: 404 → 409 transition → 403 role → side effects → record and commit. - In `create_donation`, note that ranking runs immediately and that **`MATCHED` assigns nobody**. - Note `_loaded` and `selectinload` — the N+1 answer.

You can now walk through the most complex function in the project.

■ 1:10–1:30 — The matcher

Open `code/foodlink/matching.py`. - The three hard gates in `score_pair` — and why gating beats scoring low. - The five criteria and `WEIGHTS`. - **Compute the worked example in §11.2 by hand until you get 84.** - Rehearse one sentence: *"It's an explainable weighted heuristic, not machine learning, and that was deliberate."*

You can now defend the project's headline feature honestly.

■ 1:30–1:45 — The frontend seam

- `frontend/src/lib/api.ts` — read `request<T>` and the token/401 handling. Skip the wire types.
- `frontend/src/components/ProtectedRoute.tsx` — all 50 lines. **"It's UX, not security"** must be automatic.
- Skim `AuthContext`'s boot effect and the `expiring` ref guard.

- `frontend/vite.config.ts` — 15 lines; explains why dev has no CORS.

You can now explain how the two halves connect.

■ 1:45–2:00 — Weaknesses and the pitch

- Read **§8.2 (weaknesses)** and the "Biggest limitations" block in §24. Be able to name the top four **without prompting**: no migrations, the default secret key, unscoped donation reads, no CI or rate limiting.
- Read the **§20 one-minute and three-minute pitches aloud, twice.** Out loud — reading them silently does not build the fluency you need.
- Skim the §14 test list so you can say what is covered and that the frontend has no tests at all.

You can now open the conversation strongly and survive the hard questions.

If you get a third hour

- Run the tests: `.venv/Scripts/python.exe -m pytest code/tests -q`
- Start the backend and click through `/docs`, authorising with a real token
- Read `metrics.py` end to end (98 lines) — it is the payoff of the event table
- Read `conftest.py` and understand the `StaticPool` trick
- Read `admin.py`'s last-admin guard

The five sentences to have ready

1. *"Every status change appends a server-stamped row to an event table, so the metrics are derived from evidence rather than self-reported."*
2. *"The token carries a role, but the server ignores it and re-reads the user row on every request — so suspension takes effect immediately."*
3. *"The matching is an explainable weighted heuristic, not machine learning, and that was deliberate — there's no training data and a kitchen deserves to see why it was ranked."*
4. *"The lifecycle rules are data, not conditionals, so adding a state touches a known handful of places."*
5. *"My biggest gap is that there are no migrations — `create_all` can't alter existing tables, so that's the first thing I'd fix before deploying."*

Never say: "It uses AI to match donations." It uses a weighted sum. Saying otherwise is the fastest way to lose an interviewer's trust — and the honest version is a *better* answer, because you can explain every term in it.